

Mémoire de Fin d'Études

Présenté en vue de l'obtention du

DIPLÔME DE LICENCE EN SCIENCES DE L'INFORMATIQUE ET MULTIMÉDIA - LMD

Intitulé

PharmacieConnect
Plateforme web et mobile de gestion des pharmacies de garde



Organisme d'accueil : IT-STORM Consulting

Réalisé par
Ounoun Mohamed

Sous la direction de
Dr. Riheb Naili

Encadrant professionnel : Achraf BCHIRI

Soutenu publiquement le jj/juin/2026 devant le jury composé de :

Président :	Dr.	Prénom NOM	ISIMG
Membre :	Dr.	Prénom NOM	ISIMG
Membre :	Dr.	Prénom NOM	ISIMG

Dédicace

À ma famille, pour son soutien constant et sa patience tout au long de mon parcours universitaire.

À mes enseignants, pour les connaissances, les méthodes de travail et l'encadrement qu'ils m'ont transmis.

À toutes les personnes qui m'ont encouragé pendant la préparation de ce projet de fin d'études.

Remerciements / Préface

Je tiens à remercier mon encadrant académique, Teyeb, pour son suivi, ses remarques et sa disponibilité tout au long de ce projet de fin d'études. Ses conseils m'ont aidé à mieux organiser le travail, à prendre du recul sur certains choix techniques et à présenter la solution de façon plus structurée.

Je remercie également l'organisme d'accueil **IT-STORM Consulting**, ainsi que mon encadrant professionnel, **Achraf BCHIRI**, pour l'accueil, le suivi et les orientations accordés pendant la période du stage externe. Ce cadre professionnel m'a permis de relier le travail technique à un besoin réel : faciliter la consultation des pharmacies de garde à travers une application mobile.

Je remercie aussi l'équipe pédagogique de l'Institut Supérieur d'Informatique et de Multimédia de Gabès pour l'accompagnement, les remarques et les conditions de travail accordées pendant la période du projet. L'encadrement académique a permis de garder un lien clair entre les choix techniques, les objectifs du mémoire et les exigences de la formation.

J'adresse aussi mes remerciements aux enseignants de l'Institut Supérieur d'Informatique et de Multimédia de Gabès. Les connaissances acquises durant ma formation, notamment en développement web, développement mobile, bases de données, sécurité applicative et génie logiciel, ont été directement utilisées dans la réalisation de PharmacieConnect.

Enfin, je remercie ma famille et mes amis pour leur soutien et leurs encouragements. Leur présence m'a aidé à avancer avec régularité pendant les phases d'analyse, de développement, de test et de rédaction.

Résumé

Ce projet de fin d'études porte sur la conception et la réalisation de **PharmacieConnect**, une plateforme web et mobile destinée à faciliter la recherche de pharmacies, la consultation des pharmacies de garde et l'administration des données pharmaceutiques. Le sujet répond à un besoin concret : les informations de garde sont parfois dispersées, mises à jour manuellement et peu pratiques à consulter depuis un téléphone, alors qu'elles sont souvent recherchées dans des situations urgentes.

Pour traiter ce besoin, PharmacieConnect s'appuie sur trois composants complémentaires. Le backend constitue le noyau technique de la plateforme. Il utilise **FastAPI** pour regrouper l'accès aux données, les règles métier, les endpoints REST et les mécanismes de sécurité fondés sur les jetons JWT et le contrôle des rôles. Du côté de l'administration, le tableau de bord **React/Vite** offre un espace centralisé pour maintenir les pharmacies, les plannings de garde, les médicaments, les utilisateurs autorisés, les imports CSV, les indicateurs et les journaux d'audit. L'application mobile, réalisée avec **Expo** et **React Native**, se concentre sur les besoins du citoyen : rechercher une pharmacie, consulter les gardes par date, utiliser la géolocalisation et accéder au catalogue des médicaments.

La réalisation ne se limite pas aux interfaces visibles. Elle intègre aussi plusieurs aspects de génie logiciel : modélisation des données avec SQLAlchemy, validation des entrées, contrôle d'accès basé sur les rôles, journalisation des actions sensibles, tests backend et séparation entre les interfaces clientes, les services métier et la couche de persistance.

Mots-clés : pharmacie de garde, FastAPI, React, Vite, Expo, React Native, JWT, API REST, SQLAlchemy, tableau de bord, application mobile.

Abstract

This final-year project focuses on the design and implementation of **PharmacieConnect**, a web and mobile platform for pharmacy search, on-duty pharmacy schedule consultation, and pharmaceutical data administration. The project addresses a practical need : on-duty pharmacy information can be scattered, manually updated, and difficult to consult from a mobile device, especially when users need a quick answer.

PharmacieConnect relies on three complementary components. The backend is the technical core of the platform. With **FastAPI**, it brings together data access, business rules, REST endpoints, and security mechanisms based on JWT authentication and role-based authorization. On the administration side, the **React/Vite** dashboard provides a central workspace for maintaining pharmacies, duty schedules, medicines, CSV imports, analytics, and audit logs. The mobile application uses **Expo** and **React Native** and focuses on citizens' needs : searching for pharmacies, consulting on-duty schedules by date, using geolocation, and accessing the medicine catalog.

The project is not limited to visible interfaces. It also addresses several software engineering concerns, including data modeling with SQLAlchemy, input validation, role-based access control, audit logging, backend testing, and a clear separation between client interfaces, business services, and data persistence.

Keywords : on-duty pharmacy, FastAPI, React, Vite, Expo, React Native, JWT, REST API, SQLAlchemy, dashboard, mobile application.

Table des figures

1.1	Diagramme de cas d'utilisation général	10
1.2	Planning de réalisation	11
2.1	Diagramme de cas d'utilisation détaillé	19
3.1	Architecture globale de PharmacieConnect	22
3.2	Diagramme de classes	25
3.3	Diagramme de séquence : authentification JWT	26
3.4	Diagramme de séquence : recherche d'une pharmacie	27
3.5	Diagramme d'activité : import CSV contrôlé	28
3.6	Diagramme de composants	29
3.7	Diagramme de déploiement	30
4.1	Vue du tableau de bord administrateur	43
4.2	Vue de gestion des pharmacies	43
4.3	Vue mobile d'accueil et de recherche	44
4.4	Vue mobile du calendrier des gardes	44
4.5	Vue mobile du catalogue de médicaments	44
4.6	Écran mobile de l'assistant de premiers secours	44

Liste des tableaux

1.1	Cadre institutionnel du projet	6
1.2	Comparaison des solutions existantes	7
1.3	Organisation des sprints de réalisation	10
1.4	Planification technique synthétique	11
2.1	Acteurs de la plateforme PharmacieConnect	13
2.2	Besoins fonctionnels principaux	14
2.3	Description textuelle du cas d'utilisation Authentification	15
2.4	Description textuelle du cas d'utilisation Recherche d'une pharmacie	16
2.5	Description textuelle du cas d'utilisation Consultation des gardes	16
2.6	Description textuelle du cas d'utilisation Gestion des pharmacies	17
2.7	Description textuelle du cas d'utilisation Import CSV	17
2.8	Description textuelle du cas d'utilisation Gestion des utilisateurs et assistants	18
3.1	Organisation du backend	23
3.2	Organisation de l'administration web	23
3.3	Organisation de l'application mobile	24
3.4	Entités principales du modèle de données	24
3.5	Justification des choix techniques	31
4.1	Environnement de développement	35
4.2	Technologies utilisées dans le projet	36
4.3	Organisation générale du dépôt	37
4.4	Statistiques du jeu de données de l'assistant	41
4.5	Performances de l'assistant après entraînement	42
4.6	Principales intégrations API	45
4.7	Variables d'environnement principales	45
4.8	Stratégie de validation	46
4.9	Résultats de validation exécutés	46

4.10	Preuves de lancement et de build	48
4.11	Scénarios fonctionnels testés	48
4.12	Difficultés rencontrées et solutions adoptées	49
A.1	Cas d'utilisation et éléments de validation	55
A.2	Éléments visuels à insérer	56
A.3	Résultats détaillés des commandes exécutées	58
A.4	Variables d'environnement utilisées par le backend	58
A.5	Exemple de structure CSV pour les pharmacies	59
A.6	Exemple de structure CSV pour les médicaments	59

Table des matières

Dédicace	i
Remerciements / Préface	ii
Résumé	iii
Abstract	iv
Liste des figures	v
Liste des tableaux	vi
Liste des acronymes	xii
Introduction générale	1
Contexte général	1
Problématique	1
Objectifs du projet	2
Méthodologie adoptée	2
Organisation du rapport	3
1 Cadre conceptuel et étude de l'existant	4
1.1 Introduction	5
1.2 Présentation du domaine pharmaceutique	5
1.3 Contexte général	5
1.4 Cadre du stage externe et organisme d'accueil	5
1.5 Problèmes et défis	6
1.6 Étude de l'existant	6
1.7 Analyse du marché et solutions existantes	7
1.8 Critique de l'existant	7
1.9 Solution proposée : PharmacieConnect	8

1.10	Objectifs	8
1.11	Acteurs	9
1.12	Besoins fonctionnels	9
1.13	Besoins non fonctionnels	9
1.14	Méthodologie de développement	9
1.15	Diagramme de cas d'utilisation	10
1.16	Planification technique	10
1.17	Conclusion	11
2	Analyse et spécification des besoins	12
2.1	Introduction	13
2.2	Acteurs	13
2.3	Besoins fonctionnels détaillés	13
2.4	Besoins non fonctionnels	14
2.5	Règles de gestion	15
2.6	Descriptions textuelles des cas d'utilisation	15
2.6.1	Cas d'utilisation : authentification	15
2.6.2	Cas d'utilisation : recherche d'une pharmacie	16
2.6.3	Cas d'utilisation : consultation des pharmacies de garde	16
2.6.4	Cas d'utilisation : gestion des pharmacies	17
2.6.5	Cas d'utilisation : import CSV	17
2.6.6	Cas d'utilisation : gestion des utilisateurs et assistants	18
2.7	Diagramme de cas d'utilisation	18
2.8	Conclusion	20
3	Conception	21
3.1	Introduction	22
3.2	Architecture globale	22
3.3	Backend FastAPI	22
3.4	Tableau de bord React/Vite	23
3.5	Application mobile Expo/React Native	24
3.6	Modèle de données	24
3.7	Diagramme de classes	25
3.8	Diagrammes de séquence	26
3.9	Diagramme d'activité	27
3.10	Diagramme de composants et de déploiement	29
3.11	Sécurité	30
3.11.1	JWT	30
3.11.2	Contrôle d'accès basé sur les rôles (RBAC)	30

3.11.3	Hachage des mots de passe	30
3.11.4	Validation API	31
3.11.5	Variables d'environnement	31
3.12	Justification des choix techniques	31
3.13	Conclusion	32
4	Réalisation et tests	33
4.1	Introduction	35
4.2	Environnement matériel et logiciel	35
4.3	Technologies utilisées	35
4.4	Organisation du dépôt	36
4.5	Réalisation backend	37
4.5.1	Endpoints publics	37
4.5.2	Authentification et sessions	37
4.5.3	Services métier	38
4.5.4	Imports CSV	38
4.5.5	Audit et indicateurs	38
4.6	Réalisation du tableau de bord administrateur	39
4.6.1	Pages principales	39
4.6.2	Gestion des sessions côté administration	39
4.7	Réalisation application mobile	39
4.7.1	Recherche et détail des pharmacies	39
4.7.2	Carte et géolocalisation	40
4.7.3	Calendrier des gardes	40
4.7.4	Catalogue de médicaments	40
4.7.5	Paramètres, langues et thème	40
4.8	Réalisation de l'assistant conversationnel de premiers secours	40
4.8.1	Architecture générale	40
4.8.2	Données d'apprentissage et nettoyage	41
4.8.3	Entraînement et métriques de performance	41
4.8.4	API et intégration avec l'application mobile	42
4.8.5	Tests et validation	43
4.9	Captures d'écran	43
4.10	Intégration API	44
4.11	Déploiement et configuration	45
4.12	Tests unitaires, API et manuels	46
4.12.1	Stratégie de test	46
4.12.2	Résultats de validation automatisée	46
4.12.3	Tests automatisés du backend	47

4.12.4 Tests API	47
4.12.5 Validation du déploiement local	47
4.12.6 Validation fonctionnelle	48
4.13 Difficultés rencontrées et solutions adoptées	49
4.14 Conclusion	49
Conclusion générale	50
Bilan	50
Résultats obtenus	50
Limites	51
Perspectives	51
Bibliographie	53
A Annexes	55
A.1 Descriptions complémentaires des cas d'utilisation	55
A.2 État des captures et diagrammes	55
A.3 Commandes de lancement	56
A.4 Commandes de test	57
A.5 Résultats de validation du 13 mai 2026	58
A.6 Variables d'environnement	58
A.7 Exemples CSV	59

Liste des acronymes

API Application Programming Interface.

CSV Comma-Separated Values.

GPS Global Positioning System.

HTTP HyperText Transfer Protocol.

ISIMG Institut Supérieur d'Informatique et de Multimédia de Gabès.

JWT JSON Web Token.

ORM Object-Relational Mapping.

PCT Pharmacie Centrale de Tunisie.

PFE Projet de Fin d'Études.

RBAC Role-Based Access Control.

REST Representational State Transfer.

RTL Right-To-Left.

SQL Structured Query Language.

UML Unified Modeling Language.

URL Uniform Resource Locator.

UX User Experience.

Introduction générale

Contexte général

Dans le domaine de la santé, l'accès à une information fiable peut avoir un impact direct sur le service rendu au citoyen. Pour les pharmacies, cette information concerne notamment l'adresse, le numéro de téléphone, les horaires et les périodes de garde. Lorsqu'un utilisateur cherche une pharmacie disponible, il attend une réponse simple et rapide, surtout en dehors des horaires habituels.

En Tunisie, les informations relatives aux pharmacies de garde existent, mais leur consultation reste parfois peu pratique. Elles peuvent être diffusées sous forme de listes, relayées localement ou mises à jour manuellement. Cette dispersion complique la recherche depuis un téléphone, en particulier lorsque l'utilisateur ne connaît pas bien la ville ou le gouvernorat concerné.

Le projet **PharmacieConnect** s'inscrit dans ce contexte. Il associe une API backend FastAPI, un tableau de bord administrateur React/Vite, une application mobile Expo/React Native et un assistant conversationnel spécialisé en premiers secours. Le backend porte la logique commune, tandis que les interfaces répondent à des usages complémentaires : la consultation citoyenne, l'administration des données et l'assistance médicale d'urgence. L'assistant conversationnel enrichit l'application mobile en fournissant des réponses rapides et fiables aux questions de premiers secours, complément naturel au repérage des pharmacies de garde.

Problématique

La problématique du projet peut être formulée ainsi : comment concevoir une plateforme qui permette au citoyen de trouver rapidement une pharmacie ou une pharmacie de garde, tout en offrant à l'administrateur un moyen fiable de gérer, corriger et suivre les données ?

Cette question impose plusieurs contraintes. L'application mobile doit rester

simple à utiliser et fournir des résultats lisibles. Le tableau de bord doit faciliter l'import, la correction et le suivi des données. Le backend, quant à lui, doit contrôler les accès, valider les informations reçues et conserver une trace des actions importantes.

Objectifs du projet

Le projet vise à réaliser une plateforme complète autour des pharmacies et des pharmacies de garde, enrichie par un assistant de premiers secours. Les objectifs retenus sont les suivants :

- concevoir une API backend centralisant les données et les règles métier ;
- mettre en place une authentification sécurisée avec des jetons JSON Web Token (JWT) ;
- mettre à disposition une interface web pour gérer les pharmacies, les gardes, les médicaments et les utilisateurs autorisés ;
- réaliser une application mobile orientée vers la recherche de pharmacies, la consultation des gardes et l'accès au catalogue de médicaments ;
- intégrer un assistant conversationnel spécialisé en premiers secours utilisant la récupération d'informations hybride ;
- intégrer une recherche par proximité à partir des coordonnées Global Positioning System (GPS) ;
- prendre en charge l'import de fichiers Comma-Separated Values (CSV) avec validation des données et retour d'erreurs ;
- ajouter des journaux d'audit, des indicateurs et un contrôle d'accès basé sur les rôles ;
- tester les fonctionnalités principales, y compris l'assistant et ses mécanismes de sécurité, à travers des tests backend et des scénarios fonctionnels.

Méthodologie adoptée

La démarche suivie est itérative et incrémentale, avec une organisation inspirée de Scrum [1]. Le travail a été découpé en cycles courts afin d'avancer progressivement sur des modules vérifiables : analyse du besoin, étude de l'existant, définition des fonctionnalités, conception de l'architecture, développement du backend, développement de l'administration web, développement de l'application mobile, intégration, tests et rédaction du rapport.

La modélisation s'appuie sur le formalisme Unified Modeling Language (UML) [2] pour représenter les acteurs, les cas d'utilisation, les classes principales et les échanges

entre les composants. La réalisation a d'abord porté sur les règles métier du backend, puisque l'API centralise la validation, la sécurité et l'accès aux données. Les interfaces web et mobile ont ensuite été raccordées progressivement aux endpoints disponibles.

Les fonctionnalités sensibles ont été vérifiées au fur et à mesure de leur intégration. Cette progression a facilité la validation des imports, de l'authentification, de la gestion des rôles, de la recherche par proximité et des principaux parcours utilisateur.

Organisation du rapport

Le rapport est organisé en quatre chapitres. Le premier situe le projet dans son cadre général, présente le domaine pharmaceutique et analyse l'existant. Le deuxième précise les acteurs, les besoins fonctionnels et non fonctionnels, y compris les besoins relatifs à l'assistant de premiers secours, et expose les règles de gestion. Le troisième est consacré à la conception : architecture globale, architecture de l'assistant, modèle de données, diagrammes UML et sécurité. Le quatrième décrit la réalisation technique du backend, du tableau de bord, de l'application mobile et de l'assistant, ainsi que l'intégration, la configuration, les tests et les difficultés rencontrées. Une conclusion générale, une bibliographie et des annexes complètent le document.

Chapitre 1

Cadre conceptuel et étude de l'existant

Sommaire

1.1	Introduction	5
1.2	Présentation du domaine pharmaceutique	5
1.3	Contexte général	5
1.4	Cadre du stage externe et organisme d'accueil	5
1.5	Problèmes et défis	6
1.6	Étude de l'existant	6
1.7	Analyse du marché et solutions existantes	7
1.8	Critique de l'existant	7
1.9	Solution proposée : PharmacieConnect	8
1.10	Objectifs	8
1.11	Acteurs	9
1.12	Besoins fonctionnels	9
1.13	Besoins non fonctionnels	9
1.14	Méthodologie de développement	9
1.15	Diagramme de cas d'utilisation	10
1.16	Planification technique	10
1.17	Conclusion	11

1.1 Introduction

Avant d'aborder les choix techniques, il est nécessaire de replacer PharmacieConnect dans son contexte. Le chapitre commence par le domaine pharmaceutique et le cadre du stage, puis analyse les difficultés observées dans la consultation des pharmacies de garde et les limites des solutions existantes. Il introduit enfin les objectifs du projet et les grandes lignes de la solution retenue.

1.2 Présentation du domaine pharmaceutique

La pharmacie occupe une place importante dans l'accès aux médicaments et dans l'orientation du patient. En dehors des horaires habituels, les pharmacies de garde assurent la continuité du service, notamment la nuit, les week-ends et les jours fériés.

Dans ce contexte, la qualité de l'information devient essentielle. Une adresse incorrecte, un numéro de téléphone absent ou un horaire non actualisé peut empêcher l'utilisateur de trouver la pharmacie recherchée. La mise à jour des données fait donc partie intégrante du service, au même titre que leur consultation.

1.3 Contexte général

PharmacieConnect regroupe les informations liées aux pharmacies, aux plannings de garde et aux médicaments. La plateforme s'adresse principalement à deux profils. Le citoyen utilise l'application mobile pour chercher une pharmacie, consulter les gardes ou accéder au catalogue de médicaments. L'administrateur utilise le tableau de bord web pour gérer, vérifier et mettre à jour les données.

Le backend constitue la couche centrale de la plateforme. Il regroupe la validation, la sécurité et les règles métier, tout en évitant aux interfaces clientes d'accéder directement à la base de données.

1.4 Cadre du stage externe et organisme d'accueil

Le projet a été réalisé dans le cadre d'un stage externe au sein de **IT-STORM Consulting**. Le sujet officiel concerne le développement d'une application mobile permettant aux citoyens de consulter les pharmacies de garde en Tunisie. Les principaux besoins portent sur la recherche par géolocalisation ou par saisie manuelle, l'affichage en liste et sur carte, la consultation des coordonnées et l'accès rapide aux informations utiles.

Le stage s'est déroulé du 07/02/2026 au 07/06/2026, sous l'encadrement professionnel de **Achraf BCHIRI**. Ce cadre externe a orienté le projet vers une solution

exploitable, structurée autour d'une API REST, d'une base de données, d'une application mobile et d'une interface d'administration pour la mise à jour des données.

TABLEAU 1.1 – Cadre institutionnel du projet

Élément	Description
Organisme d'accueil	IT-STORM Consulting.
Adresse	12 Rue Jean Pacilly, 91120 Palaiseau, France.
Encadrant professionnel	Achraf BCHIRI.
Période	Du 07/02/2026 au 07/06/2026.
Nature du projet	Projet de fin d'études externe orienté conception et réalisation logicielle.
Domaine	Santé numérique, accès aux pharmacies, gardes et données pharmaceutiques.
Périmètre technique	API backend, tableau de bord administrateur, application mobile et base de données.
Encadrement	Suivi professionnel, suivi académique, revues d'avancement et validation des livrables.

Ce cadre a orienté la démarche vers un prototype complet et vérifiable. La priorité a été donnée aux fonctionnalités essentielles de recherche mobile, à la séparation des responsabilités, à la fiabilité des données et à la traçabilité des actions administratives.

1.5 Problèmes et défis

Les problèmes observés concernent principalement la disponibilité, la fiabilité et l'exploitation des données. Les informations peuvent être publiées sous des formats différents, mises à jour avec retard ou présentées d'une manière peu adaptée à une recherche mobile.

Les défis retenus pour le projet sont les suivants :

- centraliser les informations sur les pharmacies et les gardes ;
- rendre la recherche simple pour un utilisateur mobile ;
- permettre la consultation par date et par proximité ;
- faciliter l'import de volumes de données à travers des fichiers CSV ;
- sécuriser l'administration et limiter les actions selon les rôles ;
- garder une trace des actions sensibles pour améliorer la traçabilité.

1.6 Étude de l'existant

Les informations sur les pharmacies de garde peuvent être consultées à travers plusieurs supports. Les listes statiques, publiées sous forme de page web, d'image ou

de document, restent simples à diffuser. Elles deviennent cependant moins pratiques lorsqu'il faut rechercher par ville, par date ou par position.

La recherche manuelle, par appels téléphoniques, réseaux sociaux ou sources locales, peut parfois donner une réponse rapide. Elle reste toutefois dépendante de la disponibilité des personnes et de la fiabilité des informations partagées.

Les applications ou sites spécialisés offrent une expérience plus adaptée, surtout lorsqu'ils proposent la géolocalisation ou la recherche par date. Leur efficacité dépend néanmoins de la qualité de la base de données, de la fréquence des mises à jour et de l'existence d'un véritable processus d'administration.

1.7 Analyse du marché et solutions existantes

Pour mieux situer PharmacieConnect, plusieurs familles de solutions ont été comparées. Les sources institutionnelles, comme le Conseil National de l'Ordre des Pharmaciens de Tunisie, constituent une référence pour les informations professionnelles et réglementaires [3]. Des annuaires médicaux comme Med.tn publient aussi des pages de pharmacies de garde, avec des informations pratiques et des coordonnées [4]. Les solutions cartographiques comme Google Maps ou OpenStreetMap facilitent la localisation d'une pharmacie, mais elles ne couvrent pas toujours le planning de garde ni les workflows d'administration métier [5, 6]. Enfin, l'application Apothicare, publiée par le Conseil national de l'ordre des pharmaciens de Tunisie, montre l'intérêt d'un accès mobile aux pharmacies ouvertes et de garde [7].

TABLEAU 1.2 – Comparaison des solutions existantes

Solution	Avantages	Limites
Sources officielles	Référence légale	Intégration app tierce
Annuaire médicaux	Recherche publique	Mises à jour externes
Cartographie	Localisation	Non spécialisée
Apps mobiles	Accès mobile	Couverture limitée
Listes statiques	Simple	Recherche manuelle
PharmacieConnect	Centralisation, audit	Alimentation régulière

1.8 Critique de l'existant

L'étude de l'existant montre que la difficulté ne se limite pas à l'affichage des données. Les informations peuvent être dispersées, présentées sous des formats différents ou mises à jour de manière irrégulière. Pour un utilisateur mobile, cette situation rend

la recherche moins fluide, surtout lorsqu'il souhaite filtrer par gouvernorat, par date ou par proximité.

La même difficulté apparaît côté administration. Lorsque le volume de données augmente, la saisie manuelle devient plus difficile à contrôler. Sans import, validation, gestion des rôles et journalisation, il devient compliqué de garder une base cohérente et de suivre les modifications effectuées.

1.9 Solution proposée : PharmacieConnect

PharmacieConnect est structurée autour de trois composants complémentaires, chacun associé à un rôle précis dans le parcours global :

- une API backend FastAPI qui centralise les données, gère l'authentification, applique les règles métier et expose les endpoints Representational State Transfer (REST) ;
- un tableau de bord administrateur React/Vite qui offre aux administrateurs un espace de gestion pour les pharmacies, les gardes, les médicaments, les imports CSV, les assistants régionaux, les indicateurs et les journaux d'audit ;
- une application mobile Expo/React Native centrée sur le citoyen, avec la recherche de pharmacies, la consultation des gardes, la géolocalisation et l'accès au catalogue de médicaments.

Cette organisation répond directement aux limites observées dans les solutions existantes, notamment la dispersion des informations, la difficulté de mise à jour et l'absence d'un workflow administratif complet. Les données sont centralisées, les accès sont contrôlés et les interfaces sont adaptées aux deux usages principaux : la consultation mobile et l'administration.

1.10 Objectifs

Les objectifs fonctionnels et techniques du projet sont :

- offrir une recherche rapide des pharmacies ;
- afficher les pharmacies de garde selon une date donnée ;
- permettre une recherche par proximité géographique ;
- administrer les pharmacies, les gardes, les médicaments et les comptes autorisés ;
- importer des fichiers CSV en contrôlant la qualité des données ;
- sécuriser les accès à travers l'authentification, les rôles et la validation côté serveur ;
- produire des journaux d'audit et des indicateurs utiles à l'administration.

1.11 Acteurs

Les acteurs principaux sont l'administrateur, l'assistant autorisé, l'utilisateur mobile et le système. Le chapitre suivant détaille leurs rôles, mais cette première identification permet déjà d'orienter la modélisation générale.

1.12 Besoins fonctionnels

Les besoins fonctionnels globaux peuvent être regroupés en quatre familles :

- besoins de consultation : recherche, filtres, détails, carte et calendrier des gardes ;
- besoins d'administration : création, modification, suppression, import et suivi des données ;
- besoins de sécurité : connexion, renouvellement de session, rôles et périmètres régionaux ;
- besoins de suivi : audit, indicateurs et contrôle des actions sensibles.

1.13 Besoins non fonctionnels

Les besoins non fonctionnels concernent la qualité attendue du système :

- **sécurité** : protection des routes administratives, hachage des mots de passe et contrôle des rôles ;
- **performance** : pagination, limitation des imports et réponses adaptées aux usages mobiles ;
- **ergonomie** : interfaces simples, lisibles et adaptées aux tâches fréquentes ;
- **maintenabilité** : séparation entre backend, tableau de bord, application mobile et base de données ;
- **fiabilité** : validation des données et gestion claire des erreurs.

1.14 Méthodologie de développement

La réalisation a suivi une démarche itérative inspirée de Scrum [1]. Dans le cadre d'un PFE, les rôles ont été adaptés : l'étudiant a assuré la conception et le développement, tandis que l'encadrement a apporté les orientations et les validations nécessaires. Chaque cycle a donné lieu à un résultat vérifiable avant de passer à l'étape suivante.

TABLEAU 1.3 – Organisation des sprints de réalisation

Sprint	Objectif	Livrables principaux
Sprint 1	Analyse et cadrage	Besoins, acteurs, règles de gestion, étude de l'existant.
Sprint 2	Conception technique	Architecture, modèle de données, sécurité, contrats API.
Sprint 3	Backend	Routes FastAPI, services métier, authentification, imports et audit.
Sprint 4	Administration web	Pages React/Vite, tableaux, formulaires, indicateurs et gestion des droits.
Sprint 5	Application mobile	Recherche, carte, gardes, médicaments, paramètres et langues.
Sprint 6	Intégration et validation	Tests, build, corrections, rapport et annexes.

Cette méthode a limité les risques liés à une intégration tardive. Les modules sensibles, notamment l'authentification, les imports CSV, les restrictions régionales et les journaux d'audit, ont été vérifiés progressivement par des tests automatisés ou des scénarios fonctionnels.

1.15 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation général met en relation les acteurs principaux avec les fonctionnalités majeures de PharmacieConnect.

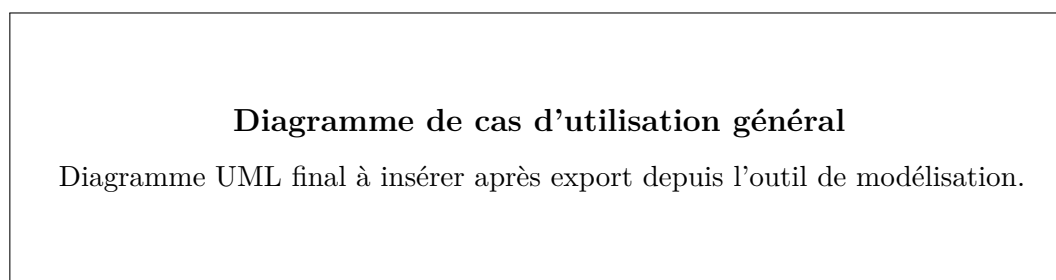


FIGURE 1.1 – Diagramme de cas d'utilisation général

1.16 Planification technique

La planification technique reprend les principales étapes de réalisation. Elle commence par l'analyse du besoin, puis enchaîne avec la conception, le développement

du backend, la réalisation des deux interfaces, l'intégration, les tests et la rédaction.

TABLEAU 1.4 – Planification technique synthétique

Phase	Travaux réalisés
Analyse	Étude du domaine, identification des acteurs et définition des besoins.
Conception	Architecture globale, modèle de données, sécurité et contrats API.
Backend	Développement FastAPI, services métier, authentification, imports et audit.
Tableau de bord	Développement React/Vite, gestion des données, indicateurs et journaux.
Mobile	Développement Expo/React Native, recherche, carte, gardes et médicaments.
Tests	Tests backend, tests API et validation manuelle des parcours principaux.
Rédaction	Structuration du rapport, annexes, bibliographie et finalisation.

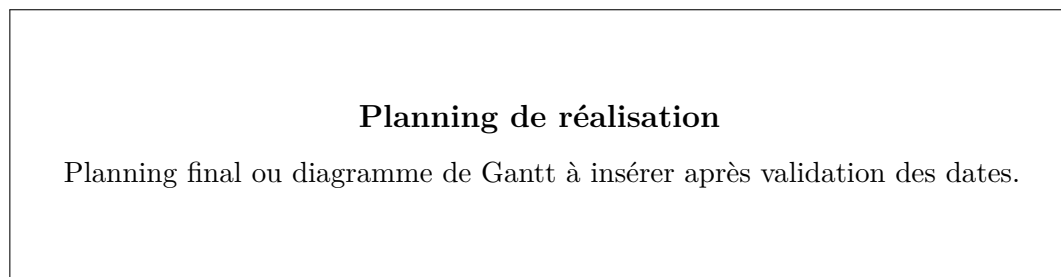


FIGURE 1.2 – Planning de réalisation

1.17 Conclusion

Ce premier chapitre a permis de préciser le contexte du projet, le domaine pharmaceutique et les limites des solutions existantes. Il montre également pourquoi PharmacieConnect repose sur une séparation entre backend, administration web et application mobile. Le chapitre suivant détaille les besoins qui découlent de ce cadrage.

Analyse et spécification des besoins

Sommaire

2.1	Introduction	13
2.2	Acteurs	13
2.3	Besoins fonctionnels détaillés	13
2.4	Besoins non fonctionnels	14
2.5	Règles de gestion	15
2.6	Descriptions textuelles des cas d'utilisation	15
2.6.1	Cas d'utilisation : authentification	15
2.6.2	Cas d'utilisation : recherche d'une pharmacie	16
2.6.3	Cas d'utilisation : consultation des pharmacies de garde	16
2.6.4	Cas d'utilisation : gestion des pharmacies	17
2.6.5	Cas d'utilisation : import CSV	17
2.6.6	Cas d'utilisation : gestion des utilisateurs et assistants	18
2.7	Diagramme de cas d'utilisation	18
2.8	Conclusion	20

2.1 Introduction

Après le cadrage général, l'analyse des besoins permet de préciser ce que la plateforme doit réellement prendre en charge. Cette étape identifie les acteurs, les fonctionnalités attendues et les règles de gestion qui guideront ensuite la conception technique.

2.2 Acteurs

La plateforme distingue plusieurs acteurs. Chacun intervient avec un rôle précis et un niveau d'accès adapté à ses responsabilités.

TABLEAU 2.1 – Acteurs de la plateforme PharmacieConnect

Acteur	Rôle dans le système
Administrateur	Gère les pharmacies, les plannings de garde, les médicaments, les imports CSV, les indicateurs, les utilisateurs autorisés et les journaux d'audit.
Assistant ou utilisateur autorisé	Intervient sur un périmètre limité, notamment pour mettre à jour les pharmacies et les gardes selon les droits accordés.
Utilisateur mobile ou citoyen	Recherche une pharmacie, consulte les pharmacies de garde, utilise la géolocalisation, consulte les médicaments et gère ses préférences.
Système	Valide les données, applique les règles métier, sécurise les accès, calcule les distances, journalise les actions et expose les API.

2.3 Besoins fonctionnels détaillés

Les besoins fonctionnels décrivent les services attendus par les utilisateurs et par l'administration. Ils sont organisés par module afin de garder une séparation claire entre consultation mobile, gestion des données et sécurité.

TABLEAU 2.2 – Besoins fonctionnels principaux

Module	Fonctionnalités attendues
Authentification	Connexion, inscription, vérification email, rafraîchissement de session, déconnexion et consultation du profil.
Recherche de pharmacies	Recherche par texte, consultation de la liste des pharmacies, détail d'une pharmacie et recherche par proximité.
Pharmacies de garde	Consultation des gardes par date, import des plannings, création, modification et suppression côté administration.
Gestion des pharmacies	Création, modification, suppression, consultation, pagination et import CSV des pharmacies.
Gestion des médicaments	Import CSV, recherche par nom commercial, DCI ou code PCT, consultation du détail et pagination.
Assistant de premiers secours	Réponse aux questions de premiers secours, récupération d'informations fiable, détection de confiance, mécanismes de sécurité médicale et fallback prudent.
Administration	Tableau de bord, gestion des assistants, contrôle des autorisations, carte administrative, indicateurs et journaux d'audit.
Application mobile	Accueil, carte, calendrier des gardes, médicaments, paramètres, langues, thème sombre, favoris et notifications.

2.4 Besoins non fonctionnels

Les besoins non fonctionnels expriment les qualités attendues de la plateforme. Ils ne correspondent pas à une page ou à une action précise, mais ils influencent directement les choix de conception et de réalisation.

- **Sécurité** : protéger les routes administratives, hacher les mots de passe, utiliser des jetons JWT, vérifier les rôles et tracer les actions sensibles.
- **Performance** : paginer les réponses, limiter les tailles d'import, éviter les réponses trop volumineuses et utiliser le cache lorsque la configuration le permet.
- **Ergonomie** : proposer une application mobile simple, une administration claire et une navigation adaptée aux opérations fréquentes.

- **Maintenabilité** : séparer les routes, services, modèles, schémas, composants et écrans afin de faciliter l'évolution du projet.
- **Disponibilité** : rendre les informations importantes accessibles à travers des endpoints publics et prévoir une configuration adaptée au déploiement.
- **Fiabilité des données** : valider les fichiers CSV, contrôler les champs obligatoires, signaler les erreurs et éviter les doublons lorsque des identifiants métier existent.

2.5 Règles de gestion

Les règles de gestion suivantes encadrent le comportement de la plateforme :

- une route d'administration ne peut être appelée que par un compte authentifié et autorisé ;
- un assistant ne peut intervenir que dans le périmètre régional qui lui est attribué ;
- les mots de passe doivent être hachés avant leur enregistrement ;
- les jetons d'accès expirés doivent être renouvelés à travers un jeton de rafraîchissement valide ;
- une pharmacie doit posséder au minimum un nom et des informations de localisation exploitables ;
- les coordonnées GPS doivent respecter les bornes valides de latitude et de longitude ;
- un planning de garde doit être associé à une date valide ;
- un fichier CSV importé doit respecter les colonnes obligatoires du module concerné ;
- les erreurs d'import doivent être retournées avec un message compréhensible ;
- les actions sensibles doivent être journalisées dans les logs d'audit.

2.6 Descriptions textuelles des cas d'utilisation

2.6.1 Cas d'utilisation : authentification

TABLEAU 2.3 – Description textuelle du cas d'utilisation Authentification

Élément	Description
Acteurs	Administrateur, assistant, utilisateur mobile
Objectif	Donner à l'utilisateur l'accès aux fonctionnalités correspondant à son profil.

Élément	Description
Précondition	L'utilisateur possède un compte valide ou suit le processus d'inscription.
Scénario nominal	L'utilisateur saisit ses identifiants, le backend vérifie les informations, génère les jetons JWT et retourne les données de session.
Postcondition	L'utilisateur est authentifié et peut accéder aux fonctionnalités autorisées.
Exceptions	Identifiants invalides, compte désactivé, email non vérifié ou jeton expiré.

2.6.2 Cas d'utilisation : recherche d'une pharmacie

TABLEAU 2.4 – Description textuelle du cas d'utilisation Recherche d'une pharmacie

Élément	Description
Acteur	Utilisateur mobile ou citoyen
Objectif	Retrouver une pharmacie à partir d'un mot-clé, d'un gouvernorat ou de la position GPS.
Précondition	L'application mobile est installée et l'API est accessible.
Scénario nominal	L'utilisateur saisit une recherche ou autorise la géolocalisation. L'application interroge l'API, puis le backend filtre les pharmacies et retourne les résultats.
Postcondition	Les pharmacies correspondantes sont affichées sous forme de liste ou sur la carte.
Exceptions	Position indisponible, recherche vide, absence de résultat ou indisponibilité temporaire de l'API.

2.6.3 Cas d'utilisation : consultation des pharmacies de garde

TABLEAU 2.5 – Description textuelle du cas d'utilisation Consultation des gardes

Élément	Description
Acteur	Utilisateur mobile ou citoyen
Objectif	Consulter les pharmacies de garde pour une date donnée.
Précondition	Des plannings de garde sont enregistrés dans la base.

Élément	Description
Scénario nominal	L'utilisateur choisit une date. L'application interroge l'API des gardes et affiche les pharmacies disponibles.
Postcondition	Les gardes correspondant à la date choisie sont affichées.
Exceptions	Date invalide, absence de planning ou erreur de communication avec l'API.

2.6.4 Cas d'utilisation : gestion des pharmacies

TABLEAU 2.6 – Description textuelle du cas d'utilisation Gestion des pharmacies

Élément	Description
Acteurs	Administrateur, assistant autorisé
Objectif	Ajouter, modifier, consulter ou supprimer les informations des pharmacies.
Précondition	L'acteur est authentifié et possède les droits nécessaires.
Scénario nominal	L'acteur ouvre la page des pharmacies et effectue l'opération souhaitée. Le backend valide les données avant d'enregistrer la modification.
Postcondition	Les données sont mises à jour et l'action est journalisée si nécessaire.
Exceptions	Données invalides, pharmacie introuvable, droits insuffisants ou périmètre régional non autorisé.

2.6.5 Cas d'utilisation : import CSV

TABLEAU 2.7 – Description textuelle du cas d'utilisation Import CSV

Élément	Description
Acteurs	Administrateur, assistant autorisé selon le périmètre
Objectif	Importer un ensemble de pharmacies, de gardes ou de médicaments à partir d'un fichier CSV.
Précondition	L'acteur est authentifié et le fichier respecte le format attendu.
Scénario nominal	L'acteur charge le fichier. Le backend vérifie l'extension, la taille, les colonnes et les valeurs, puis enregistre les lignes valides.

Élément	Description
Postcondition	Les données valides sont enregistrées et un bilan d'import est retourné.
Exceptions	Colonnes manquantes, lignes invalides, fichier trop volumineux, doublons ou données hors périmètre régional.

2.6.6 Cas d'utilisation : gestion des utilisateurs et assistants

TABLEAU 2.8 – Description textuelle du cas d'utilisation Gestion des utilisateurs et assistants

Élément	Description
Acteur	Administrateur ou super administrateur
Objectif	Créer, modifier, activer, désactiver ou limiter les comptes administratifs et assistants.
Précondition	L'acteur dispose d'un rôle autorisé.
Scénario nominal	L'administrateur renseigne les informations du compte, choisit le rôle et le périmètre éventuel, puis valide l'opération.
Postcondition	Le compte est créé ou mis à jour avec les permissions correspondantes.
Exceptions	Email déjà utilisé, rôle invalide, droits insuffisants ou informations manquantes.

2.7 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation synthétise les interactions entre les acteurs et la plateforme. Il sert de repère visuel pour relier les besoins exprimés aux fonctionnalités retenues.

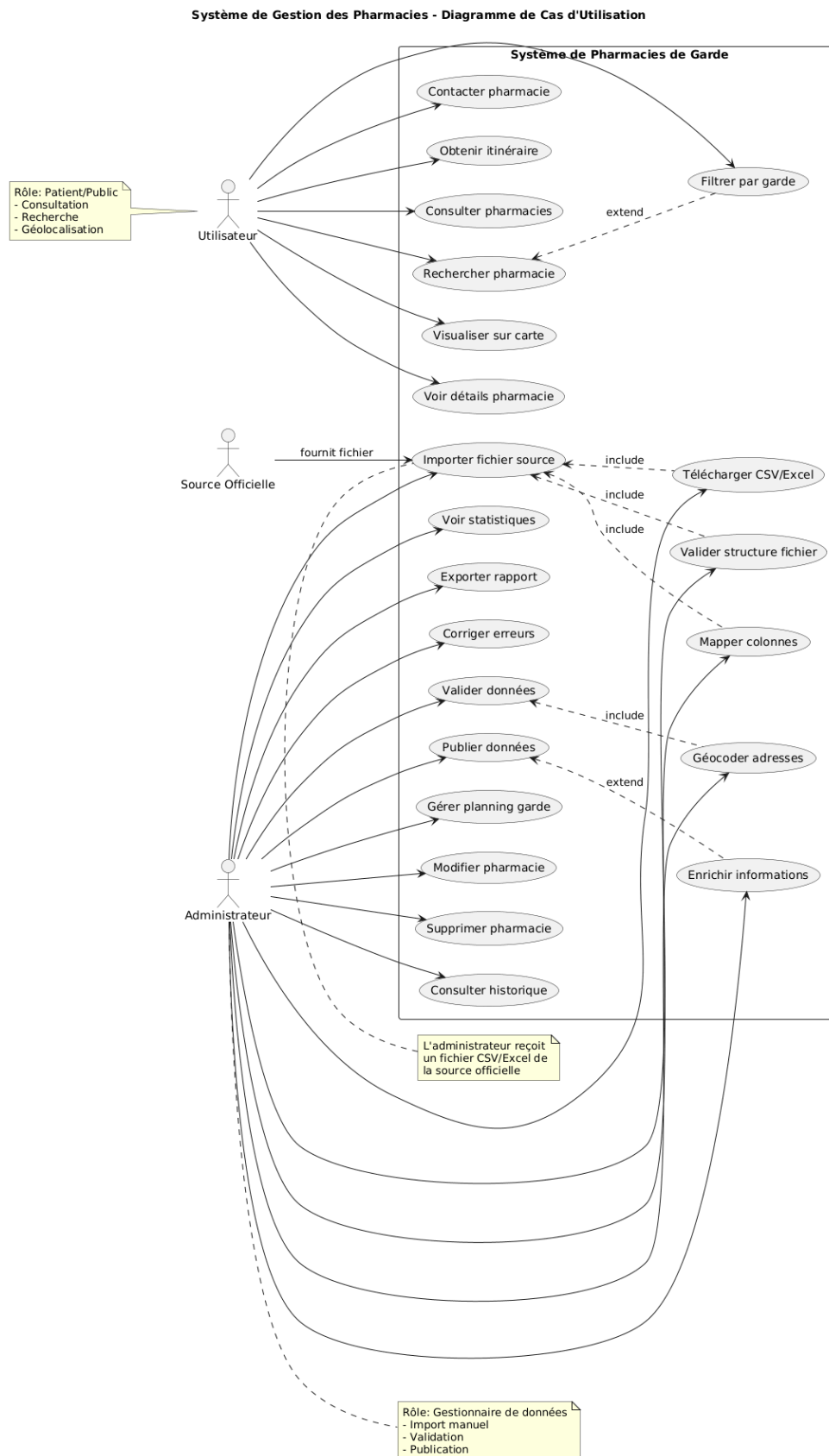


FIGURE 2.1 – Diagramme de cas d'utilisation détaillé

2.8 Conclusion

Cette analyse précise les acteurs, les besoins et les règles de gestion de PharmacieConnect. Elle établit le lien entre les attentes fonctionnelles et les choix techniques qui seront détaillés dans le chapitre de conception.

Chapitre 3

Conception

Sommaire

3.1	Introduction	22
3.2	Architecture globale	22
3.3	Backend FastAPI	22
3.4	Tableau de bord React/Vite	23
3.5	Application mobile Expo/React Native	24
3.6	Modèle de données	24
3.7	Diagramme de classes	25
3.8	Diagrammes de séquence	26
3.9	Diagramme d'activité	27
3.10	Diagramme de composants et de déploiement	29
3.11	Sécurité	30
3.11.1	JWT	30
3.11.2	Contrôle d'accès basé sur les rôles (RBAC)	30
3.11.3	Hachage des mots de passe	30
3.11.4	Validation API	31
3.11.5	Variables d'environnement	31
3.12	Justification des choix techniques	31
3.13	Conclusion	32

3.1 Introduction

La conception traduit les besoins identifiés en une organisation technique cohérente. Elle précise la répartition des responsabilités entre les composants, la structure des données, les mécanismes de sécurité et les choix techniques qui soutiennent la réalisation de PharmacieConnect.

3.2 Architecture globale

PharmacieConnect repose sur une architecture client-serveur. L'application mobile et le tableau de bord administrateur ne communiquent pas directement avec la base de données : toutes les opérations passent par l'API backend. Cette couche commune concentre la validation, la sécurité et les règles métier, ce qui évite de dupliquer ces responsabilités dans les interfaces clientes.

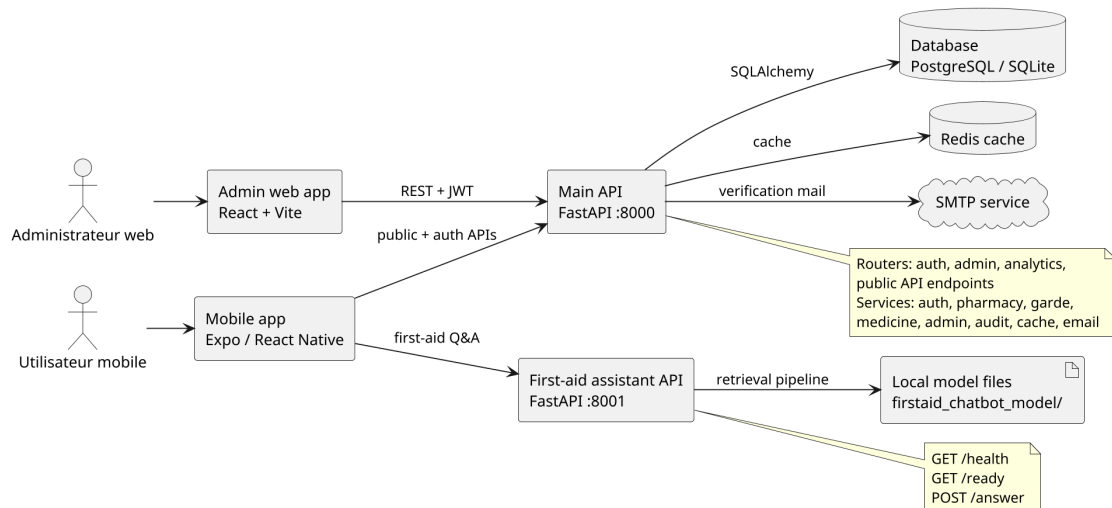


FIGURE 3.1 – Architecture globale de PharmacieConnect

L'application mobile utilise principalement les endpoints publics et les services liés au compte utilisateur. Le tableau de bord administrateur s'appuie sur des endpoints protégés pour les opérations de gestion. Le backend devient ainsi la couche commune entre les interfaces clientes et la base de données.

3.3 Backend FastAPI

Le backend s'appuie sur FastAPI pour exposer les endpoints REST et organiser le traitement des requêtes. Sa structure sépare les routes HTTP, les services métier, les modèles de données et les schémas de validation. Cette séparation rend le code plus lisible, limite le couplage entre les modules et facilite les tests.

TABLEAU 3.1 – Organisation du backend

Élément	Rôle
<code>main.py</code>	Initialisation de l'application FastAPI, middlewares, migrations au démarrage et endpoints publics.
<code>routers/</code>	Routes d'authentification, d'administration et d'indicateurs.
<code>services/</code>	Logique métier liée à l'authentification, aux pharmacies, aux gardes, aux médicaments, à l'audit et au cache.
<code>models.py</code>	Modèles SQLAlchemy représentant les tables principales.
<code>schemas.py</code>	Schémas Pydantic utilisés pour valider les entrées et structurer les sorties.
<code>migrations/</code>	Scripts SQL permettant de faire évoluer le schéma de la base.
<code>events/</code>	Bus d'événements interne et écouteurs associés.

Le traitement d'une requête suit généralement le même parcours. Le client appelle une route FastAPI ; la route vérifie les paramètres et l'utilisateur courant ; le service métier applique ensuite les règles nécessaires avant l'accès à la base via SQLAlchemy.

3.4 Tableau de bord React/Vite

Côté administration, React et Vite ont été utilisés pour construire une interface web modulaire. Les pages, les composants réutilisables, les contextes applicatifs et le client API centralisé y sont séparés afin de garder une organisation cohérente côté frontend.

TABLEAU 3.2 – Organisation de l'administration web

Élément	Rôle
<code>src/pages/</code>	Pages principales du tableau de bord : pharmacies, gardes, médicaments, carte, audit, paramètres et gestion.
<code>src/components/</code>	Composants d'interface, tableaux, formulaires, modales, cartes statistiques et éléments de layout.
<code>src/context/</code>	Gestion de l'authentification, de la langue et de certains états globaux.
<code>src/lib/api.js</code>	Client Axios responsable des appels backend et de l'injection du jeton JWT.
<code>src/styles/</code>	Styles et utilitaires liés à la présentation de l'interface.

Cette organisation évite de mélanger l’affichage avec la logique d’accès aux données. Le client API ajoute le jeton d’accès aux requêtes protégées et prend en charge le renouvellement de session lorsque le jeton expire.

3.5 Application mobile Expo/React Native

Pour la partie mobile, Expo et React Native offrent une base adaptée aux écrans destinés au citoyen. L’organisation de l’application suit les parcours attendus par l’utilisateur final : recherche, carte, calendrier de garde, médicaments, authentification et paramètres.

TABLEAU 3.3 – Organisation de l’application mobile

Élément	Rôle
screens/	Écrans de l’application : accueil, carte, calendrier, médicaments, connexion, inscription, vérification email et paramètres.
components/	Composants réutilisables comme les cartes de pharmacies, modales, sélecteurs et éléments RTL.
config/api.js	Configuration de l’URL backend et des appels API.
utils/	Fonctions utilitaires pour les données, les couleurs, le thème, le logging et la gestion RTL.
locales/	Fichiers de traduction français, anglais et arabe.

Des contextes React Native partagent certaines informations entre les écrans, comme l’authentification, la langue, le thème, les favoris, les filtres et les notifications. Cette approche évite de répéter la même logique dans plusieurs parties de l’application.

3.6 Modèle de données

Le modèle de données est défini avec SQLAlchemy. Les entités principales utilisées par le backend sont résumées dans le tableau suivant.

TABLEAU 3.4 – Entités principales du modèle de données

Entité	Description
Utilisateur	Compte utilisateur de l’application mobile avec email, mot de passe, statut actif et vérification email.
Administrateur	Compte de gestion avec rôle, statut, informations de profil et périmètre régional éventuel.

Entité	Description
RefreshToken	Jetons de rafraîchissement permettant de prolonger une session selon les règles de sécurité.
LoginAttempt	Tentative de connexion utilisée pour suivre les succès et les échecs d'authentification.
Pharmacie	Données d'une pharmacie : nom, adresse, téléphone, gouvernorat, latitude, longitude et référence OSM éventuelle.
GardeSchedule	Planning de garde avec date, pharmacie, horaire, ville, gouvernorat, type de garde et notes.
Medicine	Catalogue de médicaments avec code PCT, nom commercial, prix public, tarif de référence, remboursement, DCI et AP.
AuditLog	Trace des actions importantes avec acteur, cible, statut, adresse IP et détails.
SearchEvent	Événements de recherche utilisés pour alimenter les statistiques et les indicateurs.

3.7 Diagramme de classes

Le diagramme de classes doit représenter les entités principales, leurs attributs utiles et les relations entre les comptes, les pharmacies, les gardes, les médicaments et les journaux d'audit.

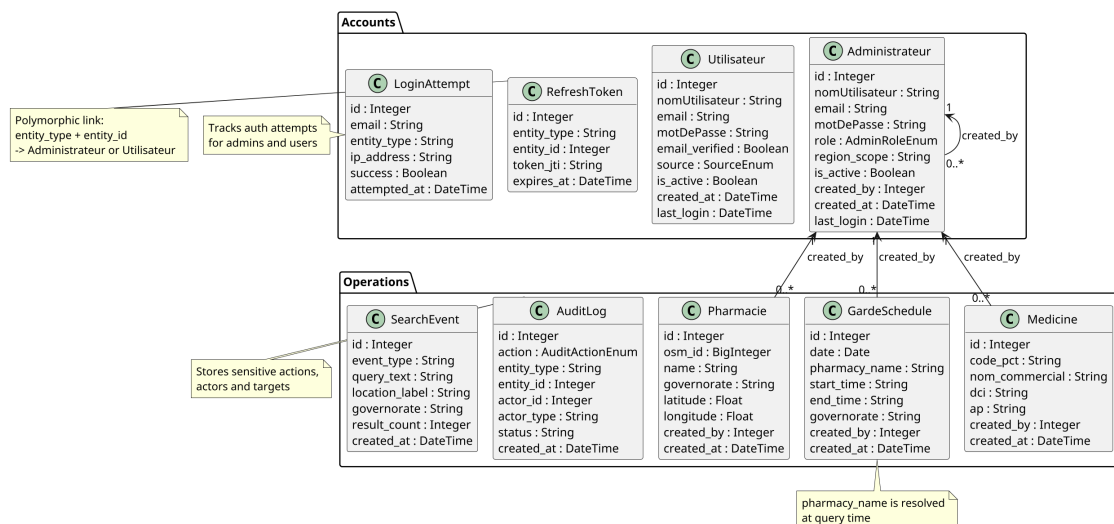


FIGURE 3.2 – Diagramme de classes

3.8 Diagrammes de séquence

Les diagrammes de séquence servent à décrire les échanges entre les interfaces clientes, le backend, les services métier et la base de données.

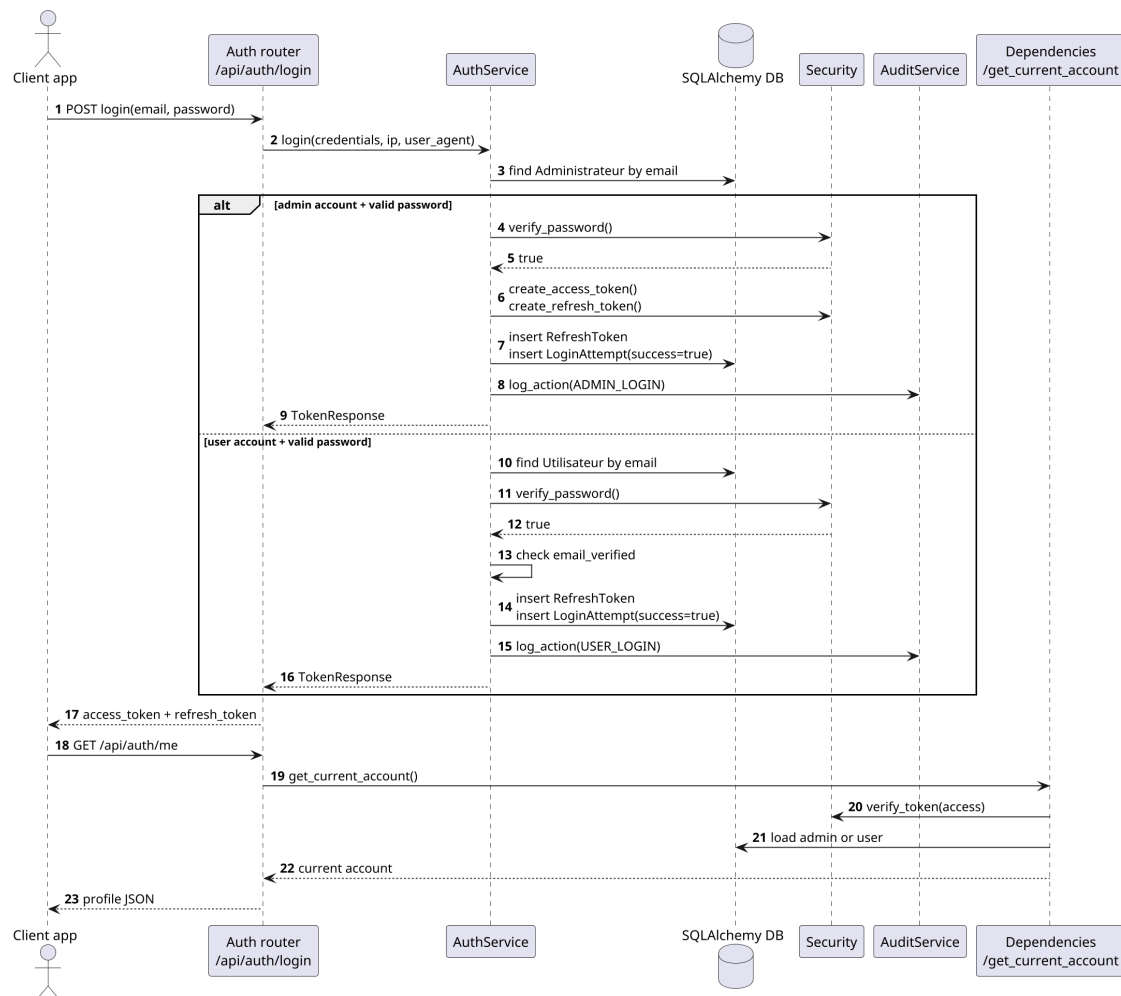


FIGURE 3.3 – Diagramme de séquence : authentification JWT

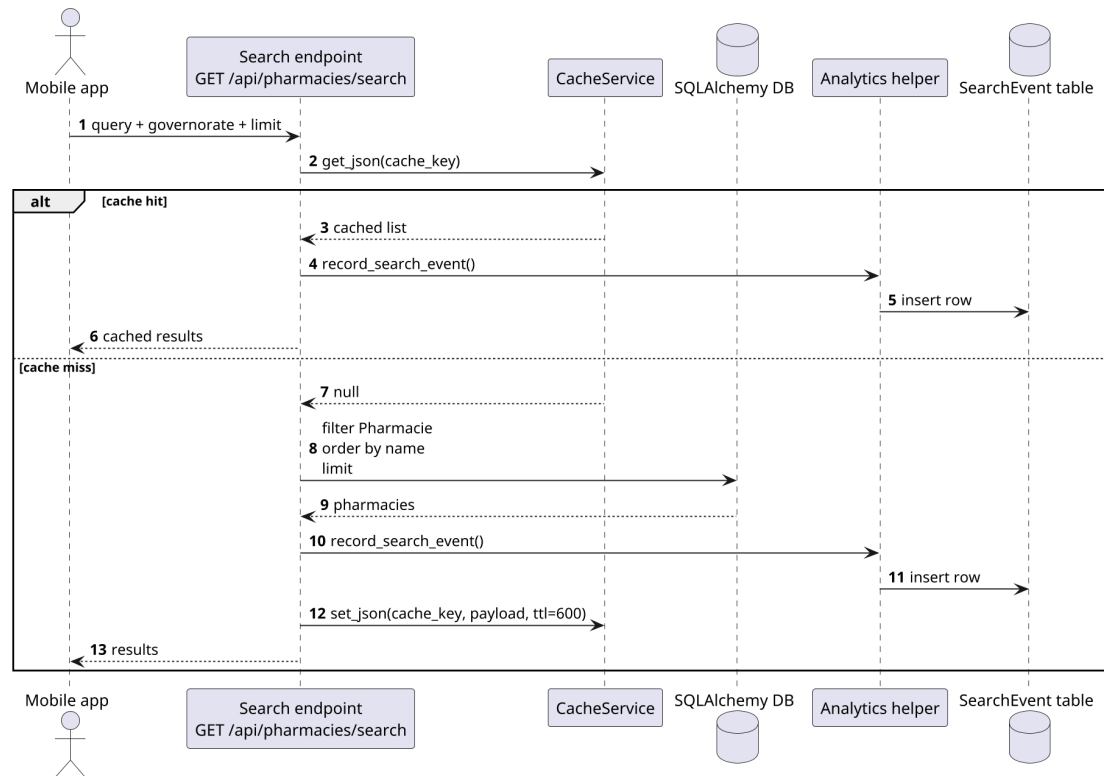


FIGURE 3.4 – Diagramme de séquence : recherche d’une pharmacie

3.9 Diagramme d’activité

Le diagramme d’activité prévu détaille le traitement d’un import CSV. Ce scénario est important, car il regroupe plusieurs règles sensibles : contrôle du fichier, validation des colonnes, vérification des lignes, enregistrement des données valides et journalisation du résultat.

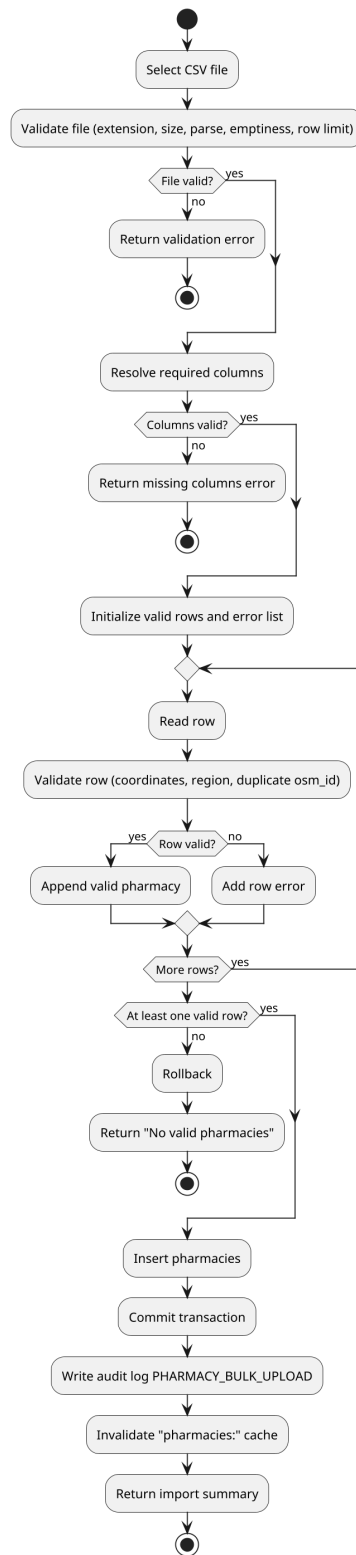


FIGURE 3.5 – Diagramme d’activité : import CSV contrôlé

3.10 Diagramme de composants et de déploiement

Les diagrammes de composants et de déploiement complètent la vision technique. Le premier décrit les modules logiciels, tandis que le second présente la répartition logique entre les clients, le serveur backend, la base de données et la configuration.

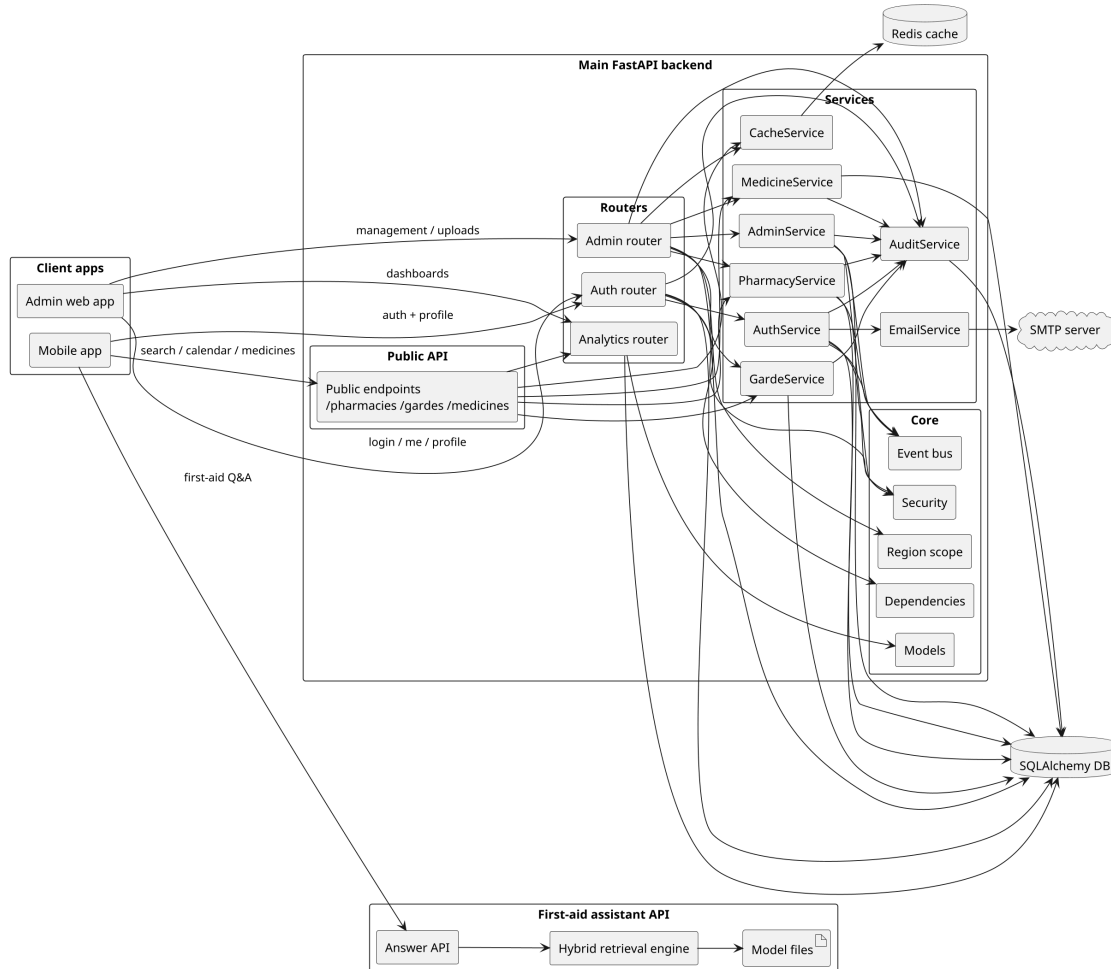


FIGURE 3.6 – Diagramme de composants

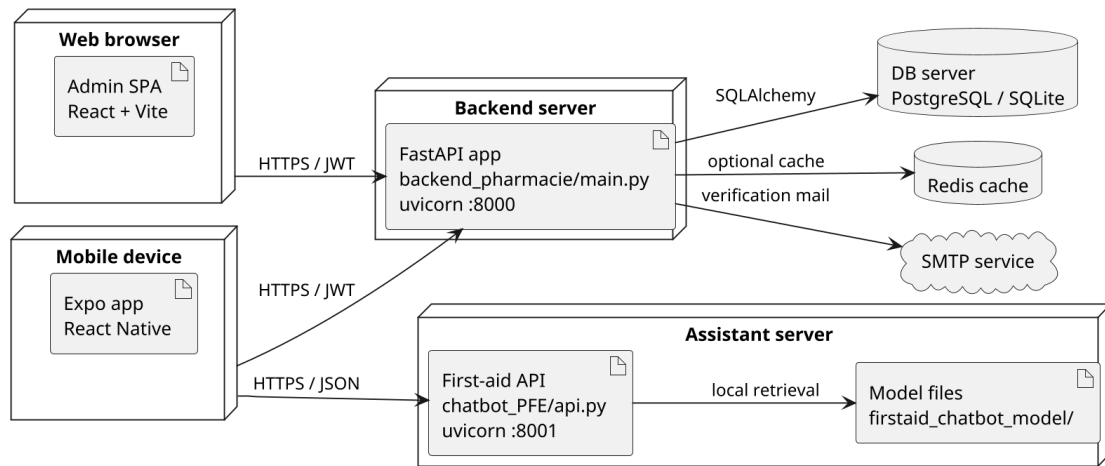


FIGURE 3.7 – Diagramme de déploiement

3.11 Sécurité

La sécurité est traitée au niveau du backend, car les règles d'accès ne doivent pas dépendre uniquement des interfaces clientes. Les mécanismes retenus suivent les principes généraux de protection des API et de réduction des accès non autorisés [8].

3.11.1 JWT

Après une connexion réussie, le backend génère un jeton d'accès et un jeton de rafraîchissement. Le jeton d'accès autorise l'appel des routes protégées, tandis que le jeton de rafraîchissement renouvelle la session lorsque le premier expire. Le format JWT est défini par la RFC 7519 [9].

3.11.2 Contrôle d'accès basé sur les rôles (RBAC)

La plateforme distingue plusieurs rôles administratifs, notamment administrateur, super administrateur et assistant. Ces rôles limitent les actions accessibles. Les assistants peuvent aussi être liés à un périmètre régional afin de limiter leurs interventions aux zones autorisées.

3.11.3 Hachage des mots de passe

Les mots de passe ne sont pas stockés en clair. Le backend les hache avant l'enregistrement et compare les valeurs hachées pendant la connexion. Ce choix réduit l'impact d'un éventuel accès non autorisé à la base de données.

3.11.4 Validation API

Les données envoyées vers l'API sont validées avec Pydantic et avec les services métier. Cette validation concerne les champs obligatoires, les coordonnées GPS, les dates, les fichiers CSV, les limites de pagination et les autorisations.

3.11.5 Variables d'environnement

Les paramètres sensibles sont placés dans des variables d'environnement. Cela concerne notamment la chaîne de connexion à la base de données, la clé de signature JWT, la durée des jetons, les origines CORS, les hôtes autorisés et la configuration du cache.

3.12 Justification des choix techniques

TABLEAU 3.5 – Justification des choix techniques

Choix	Justification
FastAPI	Framework Python adapté aux API REST, avec validation Pydantic et documentation OpenAPI automatique [10].
React et Vite	Ensemble adapté à la création d'une interface d'administration rapide, modulaire et maintenable [11, 12].
Expo et React Native	Permettent de développer une application mobile avec une base de code JavaScript et un accès simplifié aux fonctionnalités natives [13, 14].
SQLAlchemy	ORM permettant de structurer l'accès aux données tout en conservant un contrôle sur les modèles et les requêtes [15].
SQLite et PostgreSQL	SQLite facilite le développement local, tandis que PostgreSQL constitue une option plus adaptée à un environnement configuré [16].
JWT	Mécanisme adapté à la sécurisation des API et à la séparation entre jeton d'accès et jeton de rafraîchissement.
CSV	Format simple pour importer des volumes de données de pharmacies, gardes et médicaments.

3.13 Conclusion

La conception retenue sépare clairement le backend, l'administration web, l'application mobile et la base de données. Cette organisation facilite le développement, les tests et les évolutions futures. Le chapitre suivant passe de cette conception à la réalisation technique et à la validation de la solution.

Chapitre 4

Réalisation et tests

Sommaire

4.1	Introduction	35
4.2	Environnement matériel et logiciel	35
4.3	Technologies utilisées	35
4.4	Organisation du dépôt	36
4.5	Réalisation backend	37
4.5.1	Endpoints publics	37
4.5.2	Authentification et sessions	37
4.5.3	Services métier	38
4.5.4	Imports CSV	38
4.5.5	Audit et indicateurs	38
4.6	Réalisation du tableau de bord administrateur	39
4.6.1	Pages principales	39
4.6.2	Gestion des sessions côté administration	39
4.7	Réalisation application mobile	39
4.7.1	Recherche et détail des pharmacies	39
4.7.2	Carte et géolocalisation	40
4.7.3	Calendrier des gardes	40
4.7.4	Catalogue de médicaments	40
4.7.5	Paramètres, langues et thème	40
4.8	Réalisation de l'assistant conversationnel de premiers secours	40
4.8.1	Architecture générale	40
4.8.2	Données d'apprentissage et nettoyage	41
4.8.3	Entraînement et métriques de performance	41
4.8.4	API et intégration avec l'application mobile	42
4.8.5	Tests et validation	43

4.9 Captures d'écran	43
4.10 Intégration API	44
4.11 Déploiement et configuration	45
4.12 Tests unitaires, API et manuels	46
4.12.1 Stratégie de test	46
4.12.2 Résultats de validation automatisée	46
4.12.3 Tests automatisés du backend	47
4.12.4 Tests API	47
4.12.5 Validation du déploiement local	47
4.12.6 Validation fonctionnelle	48
4.13 Difficultés rencontrées et solutions adoptées	49
4.14 Conclusion	49

4.1 Introduction

Après la conception, cette partie décrit la réalisation concrète de PharmacieConnect. Elle présente l’environnement de travail, les technologies utilisées, l’organisation du dépôt et les principales fonctionnalités développées dans le backend, le tableau de bord administrateur et l’application mobile. La fin du chapitre est consacrée à l’intégration, à la configuration, aux tests et aux difficultés rencontrées.

4.2 Environnement matériel et logiciel

Le développement de PharmacieConnect a mobilisé plusieurs outils, car le projet combine un backend, une interface web d’administration et une application mobile.

TABLEAU 4.1 – Environnement de développement

Élément	Description
Machine de développement	Ordinateur portable Intel Core i7-8750H, 12 threads, 16 Go RAM, stockage local 343 Go.
Système d’exploitation	Linux 6.17 x86_64.
Éditeur de code	Visual Studio Code.
Backend	Python, FastAPI, Uvicorn, SQLAlchemy, Pydantic, Pytest
Administration web	Node.js, React, Vite, React Router, Axios, Tailwind CSS
Application mobile	Expo, React Native, React Navigation, i18next
Base de données	SQLite en développement local, PostgreSQL selon la configuration
Outils de test	Pytest, Jest, build Vite et tests fonctionnels manuels

4.3 Technologies utilisées

Les choix techniques découlent directement des besoins du projet. Le backend devait fournir des API claires et sécurisées. L’administration nécessitait une interface web capable de gérer les données depuis un navigateur. L’application mobile, de son côté, devait rester simple à utiliser tout en intégrant la recherche, la carte et la consultation des gardes.

TABLEAU 4.2 – Technologies utilisées dans le projet

Technologie	Utilisation
FastAPI	Développement de l'API REST, endpoints publics et endpoints protégés.
SQLAlchemy	Modélisation des entités et accès à la base de données.
Pydantic	Validation des données entrantes et structuration des réponses.
JWT	Authentification, jetons d'accès et jetons de rafraîchissement.
React	Construction de l'interface d'administration.
Vite	Outil de développement et de build pour le tableau de bord administrateur.
Axios	Communication entre le tableau de bord administrateur et l'API.
Tailwind CSS	Mise en forme de l'interface d'administration.
Expo	Développement et exécution de l'application mobile.
React Native	Création des écrans et composants mobiles.
i18next	Gestion de l'internationalisation.
OpenStreetMap / Nominatim	Référence cartographique et géocodage lorsque l'enrichissement des coordonnées est nécessaire [17].
BM25 (Okapi)	Algorithme de récupération lexicale pour l'assistant de premiers secours.
Sentence Transformers	Modèle d'embeddings sémantiques (<code>all-MiniLM-L6-v2</code>) pour la récupération dense.
Retrieval Pipeline	Pipeline hybride de fusion lexicale/sémantique avec reclassement et extraction de réponses.
Pytest	Tests automatisés du backend et de l'assistant [18].
Jest	Tests automatisés côté application mobile [19].

4.4 Organisation du dépôt

Le projet est regroupé dans un même espace de travail. Cette structure a facilité l'intégration entre l'API, le tableau de bord et l'application mobile, notamment lors des tests de communication entre les clients et le backend.

TABLEAU 4.3 – Organisation générale du dépôt

Dossier	Contenu
backend	API FastAPI, modèles, services, migrations
admin	Tableau de bord React/Vite
mobile	Application mobile Expo/React Native
chatbot	Assistant avec pipeline hybride
docs	Documentation technique

4.5 Réalisation backend

Le backend constitue la partie centrale de PharmacieConnect. Il reçoit les requêtes des deux interfaces, applique les règles métier et interagit avec la base de données. Il regroupe également les mécanismes d'authentification, de validation, d'audit et d'import.

4.5.1 Endpoints publics

Les endpoints publics répondent surtout aux besoins de l'application mobile. Ils donnent accès aux informations de consultation sans passer par l'espace d'administration.

- GET `/health` pour vérifier l'état du backend ;
- GET `/api/pharmacies` pour consulter les pharmacies avec pagination ;
- GET `/api/pharmacies/search` pour rechercher par texte ;
- GET `/api/pharmacies/nearby` pour rechercher les pharmacies proches d'une position GPS ;
- GET `/api/pharmacies/{pharmacy_id}` pour consulter le détail d'une pharmacie ;
- GET `/api/gardes` pour consulter les gardes d'une date donnée ;
- GET `/api/medicines` pour rechercher dans le catalogue de médicaments ;
- GET `/api/medicines/{code_pct}` pour consulter le détail d'un médicament.

Pour la recherche par proximité, l'API utilise les coordonnées GPS envoyées par le client. Elle calcule la distance avec les pharmacies disponibles et retourne les résultats triés par distance.

4.5.2 Authentification et sessions

L'authentification est gérée côté backend afin de garder le contrôle sur les comptes et les autorisations. Lorsqu'une connexion réussit, le serveur retourne un jeton d'accès et un jeton de rafraîchissement.

Listing 4.1 – Structure simplifiée d’une réponse de connexion

```
{  
  "access_token": "[jeton d'accès]",  
  "refresh_token": "[jeton de rafraichissement]",  
  "token_type": "bearer",  
  "expires_in": 1800  
}
```

Les routes protégées vérifient ensuite le jeton reçu et le rôle de l'utilisateur. Cette vérification est réalisée côté serveur afin qu'un simple contrôle d'interface ne puisse pas être contourné.

4.5.3 Services métier

La logique principale est placée dans des services. Ce choix évite de concentrer trop de code dans les routes et rend les traitements plus faciles à tester.

- **AuthService** pour l'inscription, la connexion, la vérification email, le rafraîchissement et la déconnexion ;
- **PharmacyService** pour la gestion et l'import des pharmacies ;
- **GardeService** pour la gestion et l'import des plannings de garde ;
- **MedicineService** pour l'import, la recherche et la consultation des médicaments ;
- **AuditService** pour la journalisation des actions importantes ;
- **CacheService** pour l'utilisation du cache lorsque la configuration le permet.

4.5.4 Imports CSV

Les imports CSV ont été ajoutés pour éviter la saisie manuelle d'un grand nombre de lignes. Avant l'enregistrement, le backend vérifie l'extension, la taille, les colonnes obligatoires, le format des valeurs et les contraintes métier.

Lorsqu'une ligne est incorrecte, la réponse indique les informations nécessaires pour corriger le fichier. Pour les médicaments, le code PCT sert d'identifiant pour mettre à jour un enregistrement existant au lieu de créer un doublon.

4.5.5 Audit et indicateurs

Les actions sensibles sont enregistrées dans les journaux d'audit. Cela concerne notamment les connexions, les échecs de connexion, les inscriptions, les créations administratives et les imports. Les données statistiques alimentent le tableau de bord et donnent une vision synthétique de l'activité.

4.6 Réalisation du tableau de bord administrateur

Le tableau de bord administrateur rassemble les fonctions de gestion dans une interface web destinée aux administrateurs et aux utilisateurs autorisés.

4.6.1 Pages principales

Les principales pages développées sont :

- le tableau de bord pour les compteurs, indicateurs et activités récentes ;
- la page des pharmacies pour consulter, créer, modifier, supprimer et importer les pharmacies ;
- la page des gardes pour gérer les plannings de garde ;
- la page des médicaments pour importer et consulter le catalogue ;
- la carte pour visualiser les pharmacies géographiquement ;
- la page de management pour gérer les assistants et les autorisations ;
- la page des journaux d’audit pour consulter les actions enregistrées ;
- les pages de profil, paramètres, notifications et langues.

4.6.2 Gestion des sessions côté administration

Le tableau de bord utilise un client API centralisé. Ce client ajoute le jeton JWT aux requêtes protégées. Si le jeton d’accès expire, il tente de renouveler la session. En cas d’échec, l’utilisateur est redirigé vers la page de connexion.

4.7 Réalisation application mobile

L’application mobile correspond à la partie utilisée par le citoyen. Elle regroupe la recherche, la consultation des gardes, la carte, le catalogue de médicaments et les paramètres.

4.7.1 Recherche et détail des pharmacies

Depuis l’écran d’accueil, l’utilisateur peut rechercher une pharmacie par texte et consulter les résultats. La fiche de détail affiche les informations disponibles, comme le nom, l’adresse, le téléphone, le gouvernorat et les coordonnées lorsque celles-ci existent.

4.7.2 Carte et géolocalisation

L’application peut demander l’autorisation d’utiliser la position de l’utilisateur. Cette position est envoyée au backend afin de récupérer les pharmacies proches. La carte affiche ensuite les résultats sous forme de marqueurs.

4.7.3 Calendrier des gardes

Le calendrier donne à l’utilisateur le choix d’une date et affiche les pharmacies de garde correspondantes. Cette fonctionnalité répond au besoin principal du projet : trouver rapidement une pharmacie disponible à une date précise.

4.7.4 Catalogue de médicaments

L’écran des médicaments prend en charge la recherche par nom commercial, DCI ou code PCT. La fiche de détail affiche les informations présentes dans la base, comme le prix public, le tarif de référence et la catégorie de remboursement.

4.7.5 Paramètres, langues et thème

L’application prend en charge le français, l’anglais et l’arabe. Le choix de l’arabe active l’affichage RTL. Les paramètres regroupent aussi le thème, les notifications, les favoris et certaines préférences utilisateur.

4.8 Réalisation de l’assistant conversationnel de premiers secours

L’assistant conversationnel constitue une valeur ajoutée importante de la plateforme PharmacieConnect. Il fournit des réponses rapides, fiables et sûres aux questions de premiers secours, complétant harmonieusement la recherche de pharmacies de garde. L’assistant repose sur une architecture de récupération d’informations hybride (lexicale et sémantique) plutôt que sur la génération, afin de garantir des réponses basées sur des preuves et d’éliminer les risques d’hallucination.

4.8.1 Architecture générale

L’architecture de l’assistant suit un pipeline en plusieurs étapes :

1. **Analyse de la requête** : Normalisation, tokenisation, détection de type et sujet.
2. **Récupération parallèle** : Deux algorithmes complémentaires :
 - **BM25 (Okapi)** : Récupération lexicale basée sur fréquence des termes ;

- **Embeddings sémantiques** : Modèle `all-MiniLM-L6-v2` pour capturer le sens.
- 3. **Fusion hybride** : Normalisation et fusion des scores :
 - 0.55 pour BM25 (lexical) ;
 - 0.35 pour embeddings sémantiques ;
 - 0.10 pour bonus de correspondance thématique.
- 4. **Reclassement et extraction** : Reclassement par similarité, alignement thématique et couverture. Réponse extractive composée du texte récupéré.
- 5. **Contrôle de confiance** : Calcul multi-signaux (score, similarité, cohérence). Fallback sur confiance insuffisante.
- 6. **Sécurité médicale** : Avertissement auto pour sujets à risque (étouffement, choc, hémorragie).

4.8.2 Données d'apprentissage et nettoyage

Le jeu de données initial repose sur FirstAidQA, enrichi et nettoyé pour améliorer la qualité. Les processus de qualification et d'augmentation sont décrits ci-après.

TABLEAU 4.4 – Statistiques du jeu de données de l'assistant

Métrique	Valeur
Questions/réponses brutes	3,914 paires
Documents indexés (chunks)	5,270 segments
Sources uniques	3,898 documents
Sujets (catégories)	13 catégories (étouffement, saignements, brûlures, etc.)
Ensemble d'entraînement nettoyé	3,901 paires
Ensemble de validation nettoyé	409 paires
Changements de labels (audit)	1,841 sur l'entraînement, 165 sur la validation

Le nettoyage des données a utilisé un audit automatisé (`data_quality.py`) qui détecte et corrige les étiquettes erronées assignées d'après les termes de réponse, préserve les étiquettes originales dans `original_topic` pour examen ultérieur, et génère un rapport d'audit détaillé avec traces de modification. L'augmentation ciblée de données a renforcé les sujets faibles (premiers secours généraux, saignements, morsures) en générant 60 exemples synthétiques pour améliorer la couverture et la robustesse.

4.8.3 Entraînement et métriques de performance

L'entraînement est piloté par le script `train_firstaid_chatbot.py`, qui construit le pipeline complet et évalue les performances selon plusieurs métriques.

TABLEAU 4.5 – Performances de l’assistant après entraînement

Métrique	Valeur	Cible	Interprétation
Hit Rate / Recall@1	79.71%	$\geq 75\%$	Juste au rang 1
Recall@3	90.95%	-	Juste dans top-3
Recall@5	94.38%	$\geq 90\%$	Juste dans top-5
MRR	0.8627	≥ 0.80	Rang moyen inverse
MAP@5	0.8572	≥ 0.78	Précision moyenne
Evidence F1	0.2990	≥ 0.35	Extraction de span
Fallback Rate	2.69%	$\leq 5\%$	Refus confiance faible
Smoke Tests	14/14	100%	Cas critiques

Analyse des résultats :

- Récupération robuste au niveau document (Recall@5 = 94.38%);
- Classement fiable (MRR = 0.8627), bons documents en haut de la liste;
- Extraction de spans un point faible (F1 = 0.2990), oracle top-5 F1 = 0.3904;
- Limitation due à l’appariement réponse-document, non à couverture de récupération;
- Fallback mesuré (2.69%) confirme bon fonctionnement du mécanisme de confiance.

4.8.4 API et intégration avec l’application mobile

L’assistant est exposé via une API FastAPI dédiée, lancée sur un port séparé (8001 par défaut) pour éviter le conflit avec l’API principale de pharmacies (8000).

Listing 4.2 – Exemple de requete a l’API de l’assistant

```
POST /answer
Content-Type: application/json

{
  "query": "Comment faire face a l'etouffement?",
  "top_k": 5
}
```

Listing 4.3 – Structure de la reponse de l’assistant

```
{
  "query": "Comment faire face a l'etouffement?",
  "answer": "Faites-le tousser vigoureusement.",
  "confidence": 0.92,
  "evidence_sources": [docid_1, docid_2],
```



```
"safety_notice": "APPEL D'URGENCE REQUIS",  
"topic": "choking"  
}
```

L'intégration côté mobile ajoute un onglet ou un écran dédié à l'assistant, permettant aux utilisateurs de poser des questions de premiers secours tout en accédant à localisateurs de pharmacies de garde. Les requêtes utilisateur sont envoyées à l'API de l'assistant, qui retourne la réponse enrichie avec confiance, sources et avertissements de sécurité.

4.8.5 Tests et validation

L'assistant bénéficie de deux suites de tests : (1) Smoke Tests avec 14 scénarios pour urgences courantes (étouffement, choc, perte de conscience), tous passant à 100% ; (2) Tests unitaires avec 3 suites couvrant normalisation, fusion hybride et confiance calculée. Le système fonctionne de manière fiable et aucune hallucination ne peut survenir (extraction basée sur texte récupéré).

4.9 Captures d'écran

Les captures réelles seront ajoutées après export depuis l'administration web et l'application mobile. Les emplacements suivants conservent la structure du rapport sans utiliser de vues synthétiques.

Vue du tableau de bord administrateur

Capture réelle à insérer après exécution de l'interface d'administration.

FIGURE 4.1 – Vue du tableau de bord administrateur

Vue de gestion des pharmacies

Capture réelle à insérer après exécution de l'interface d'administration.

FIGURE 4.2 – Vue de gestion des pharmacies

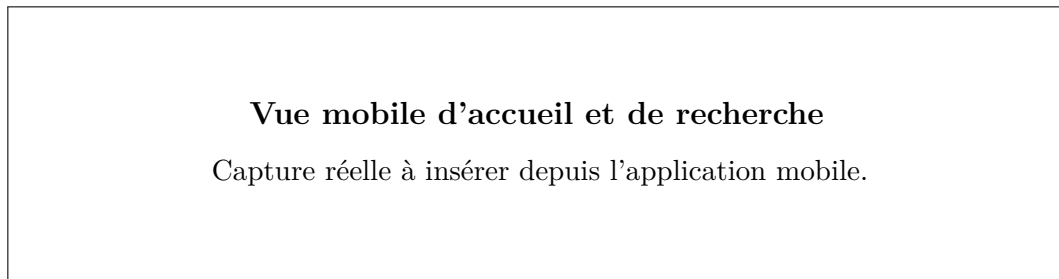


FIGURE 4.3 – Vue mobile d'accueil et de recherche

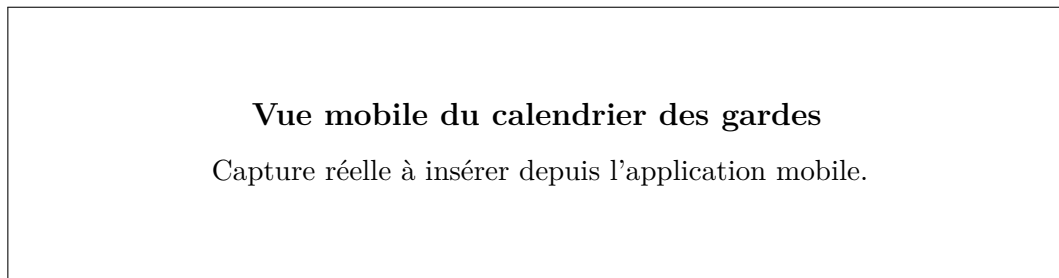


FIGURE 4.4 – Vue mobile du calendrier des gardes

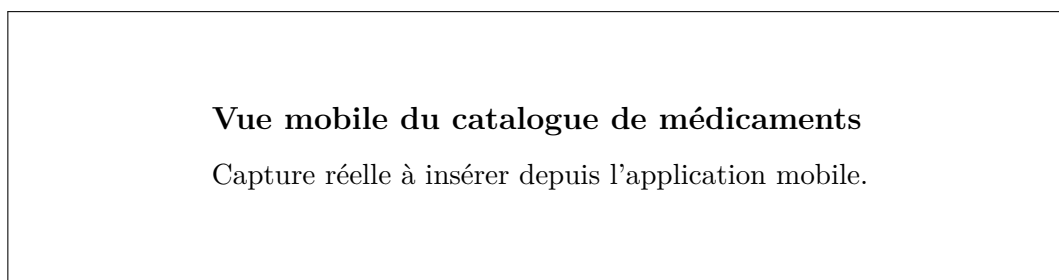


FIGURE 4.5 – Vue mobile du catalogue de médicaments

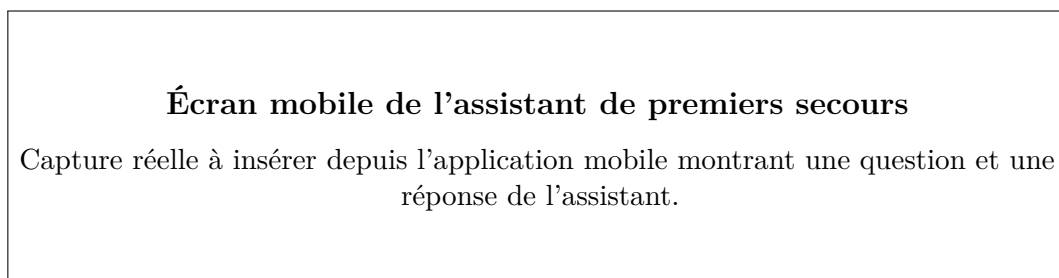


FIGURE 4.6 – Écran mobile de l'assistant de premiers secours

4.10 Intégration API

L'intégration API relie les trois interfaces (tableau de bord, application mobile et assistant) au backend. Le tableau de bord administrateur utilise Axios, tandis que

l'application mobile et l'assistant possèdent leurs propres configurations d'appel API. Les trois clients consomment les endpoints selon leurs besoins définis.

TABLEAU 4.6 – Principales intégrations API

Client	Endpoints utilisés	Objectif
Application mobile	Pharmacies, gardes, médicaments, authentification	Consultation et parcours citoyen
Tableau de bord admin	Administration, imports, indicateurs, audit	Gestion opérationnelle
Assistant QA	Récupération et classement de documents	Urgences médicales

4.11 Déploiement et configuration

La configuration du backend repose sur des variables d'environnement. Cette approche sépare le code des valeurs qui changent selon l'environnement, comme la base de données ou les clés de sécurité.

TABLEAU 4.7 – Variables d'environnement principales

Variable	Rôle
DATABASE_URL	Chaîne de connexion à la base de données.
SECRET_KEY	Clé utilisée pour signer les jetons JWT.
ALGORITHM	Algorithme de signature des jetons.
ACCESS_TOKEN_EXPIRE_MINUTES	Durée de validité du jeton d'accès.
REFRESH_TOKEN_EXPIRE_DAYS	Durée de validité du jeton de rafraîchissement.
FRONTEND_URL	Origine autorisée pour l'interface web.
TRUSTED_HOSTS	Liste des hôtes autorisés.
ENABLE_REDIS_CACHE	Activation ou désactivation du cache Redis.

Pour la version de validation, le projet est exécuté localement de manière contrôlée : le backend FastAPI est lancé avec Uvicorn, l'interface d'administration avec le serveur de développement Vite et l'application mobile avec Expo. Dans cette configuration, l'intégration entre les trois parties peut être vérifiée avant une mise en production. Pour un déploiement final, la même architecture peut être placée derrière un serveur HTTPS, avec PostgreSQL comme base de données persistante et des variables d'environnement séparées pour la production. Les commandes de lancement local sont placées en annexe afin de ne pas alourdir le corps du rapport.

4.12 Tests unitaires, API et manuels

4.12.1 Stratégie de test

La validation combine des tests automatisés côté backend et des vérifications manuelles sur les interfaces. Les tests backend portent surtout sur les règles sensibles : authentification, audit, restrictions régionales, indicateurs et imports.

TABLEAU 4.8 – Stratégie de validation

Type de test	Objectif
Tests unitaires backend	Vérifier une règle métier isolée dans un service.
Tests API	Vérifier les endpoints, les statuts HTTP, les réponses et les droits d'accès.
Tests d'import	Contrôler les fichiers CSV, les colonnes, les lignes invalides et les bilans d'import.
Tests de sécurité	Vérifier les rôles, la vérification email, les jetons et les accès interdits.
Tests manuels web	Valider les pages d'administration, formulaires, tableaux, cartes et imports.
Tests manuels mobile	Valider la recherche, la carte, le calendrier, les langues, les paramètres et les médicaments.

4.12.2 Résultats de validation automatisée

Une campagne de vérification a été exécutée sur les trois parties du projet le 13 mai 2026. Elle ne remplace pas une recette complète en production, mais elle confirme que les modules principaux compilent ou passent leurs tests automatisés dans l'environnement local.

TABLEAU 4.9 – Résultats de validation exécutés

Partie vérifiée	Commande	Résultat observé
Backend : indicateurs, assistants et audit	<code>pytest</code> sur les fichiers ciblés	14 tests réussis, 21 avertissements, durée 4,45 s.
Application mobile	<code>npm test - --runInBand</code>	7 tests réussis sur la recherche mobile par localisation, durée 2,781 s.
Administration web	<code>npm run build</code>	Build Vite terminé avec succès, génération du dossier <code>dist/</code> .

Partie vérifiée	Commande	Résultat observé
Import médicaments	<code>pytest</code> sur le fichier d'upload médicaments avec limite de temps	Exécution interrompue après 20 s sur le premier test ; ce point doit être stabilisé pour la validation complète de ce module.

Le build de l'administration a produit les artefacts `index.html`, CSS et JavaScript. Vite a signalé un avertissement concernant une taille de bundle JavaScript supérieure à 500 kB après minification. Cet avertissement n'empêche pas le build, mais il indique qu'un découpage dynamique des pages pourrait être ajouté dans une future optimisation.

4.12.3 Tests automatisés du backend

Les tests backend sont écrits avec Pytest. Ils utilisent une base de test afin de ne pas modifier les données de développement. Les fichiers identifiés couvrent notamment :

- `test_analytics_dashboard.py` pour les indicateurs du tableau de bord ;
- `test_assistant_regions.py` pour les restrictions régionales des assistants ;
- `test_audit_logs.py` pour la traçabilité des connexions et des actions ;
- `test_medicine_upload.py` pour l'import et la recherche de médicaments.

Les tests d'audit vérifient qu'une connexion administrateur réussie crée une entrée `ADMIN_LOGIN`, qu'un échec crée une entrée `ADMIN_LOGIN_FAILED`, et qu'une inscription crée une entrée `USER_REGISTERED`. Les tests de restrictions régionales contrôlent qu'un assistant ne peut agir que dans son périmètre autorisé. Les tests d'import de médicaments couvrent les colonnes obligatoires, la normalisation des prix, la gestion des doublons par `code_pct` et les messages d'erreur pour les lignes invalides.

4.12.4 Tests API

Les tests API vérifient les routes publiques et protégées. Ils contrôlent les paramètres requis, les limites de pagination, les erreurs de validation, les accès interdits et la cohérence des réponses JSON.

4.12.5 Validation du déploiement local

La version validée du projet est exécutable localement avec trois services séparés : backend FastAPI, administration React/Vite et application mobile Expo. Cette structure reflète l'architecture cible et donne un cadre clair pour vérifier les échanges API avant un déploiement final.

TABLEAU 4.10 – Preuves de lancement et de build

Composant	Commande de vérification	État
Backend FastAPI	<code>pytest</code> sur tests ciblés	Tests ciblés réussis.
Administration web	<code>npm run build</code>	Build de production réussi.
Application mobile	<code>npm test - --runInBand</code>	Tests Jest réussis.
Configuration	Variables d’environnement listées	Paramètres identifiés.

4.12.6 Validation fonctionnelle

Les principaux scénarios fonctionnels vérifiés sont présentés dans le tableau suivant.

TABLEAU 4.11 – Scénarios fonctionnels testés

Scénario	Étapes	Résultat attendu
Connexion administrateur	Saisir email et mot de passe, puis valider.	Accès au tableau de bord et stockage des jetons.
Recherche mobile	Saisir un nom, une adresse ou un gouvernorat.	Affichage d’une liste de pharmacies pertinentes.
Recherche par proximité	Autoriser la localisation et lancer la recherche.	Pharmacies triées par distance.
Consultation des gardes	Choisir une date dans le calendrier.	Affichage des pharmacies de garde correspondantes.
Import pharmacies	Charger un fichier CSV valide.	Données importées et bilan affiché.
Import gardes	Charger un planning CSV.	Gardes consultables côté administration et côté mobile.
Recherche médicament	Chercher par nom, DCI ou code PCT.	Résultats pertinents et détail accessible.
Audit logs	Filtrer les actions enregistrées.	Résultats paginés cohérents.
Changement de langue	Passer du français à l’arabe.	Interface traduite avec direction RTL appliquée.

4.13 Difficultés rencontrées et solutions adoptées

Plusieurs difficultés sont apparues pendant la réalisation. Elles ont principalement concerné la cohérence des données, les droits d'accès et l'intégration entre les trois parties du projet.

TABLEAU 4.12 – Difficultés rencontrées et solutions adoptées

Difficulté	Solution adoptée
Alignement entre backend, web et mobile	Centralisation des contrats API et séparation des services backend.
Qualité variable des fichiers CSV	Validation des colonnes, contrôle des types, retour des erreurs ligne par ligne et comportement d'upsert pour les médicaments.
Gestion des rôles et des assistants régionaux	Vérification des permissions côté backend et application du périmètre régional dans les services.
Géolocalisation mobile	Demande explicite des permissions, contrôle de l'activation des services de position et recherche par coordonnées.
Maintenabilité du backend	Séparation entre routes, services, modèles, schémas, sécurité et migrations.
Internationalisation mobile	Utilisation de fichiers de traduction et prise en charge de la direction RTL pour l'arabe.

4.14 Conclusion

La réalisation a abouti à une plateforme fonctionnelle composée d'une API sécurisée, d'un tableau de bord administrateur et d'une application mobile. Les tests réalisés ont couvert les parcours essentiels : authentification, recherche de pharmacies, consultation des gardes, imports CSV, gestion des rôles, audit et recherche de médicaments. Des améliorations restent possibles, notamment les tests bout en bout, les tests de charge et la validation sur un volume de données plus important.

Conclusion générale

Bilan

Ce projet de fin d'études a abouti à la réalisation de **PharmacieConnect**, une plateforme web et mobile répondant à un besoin concret : faciliter l'accès aux informations sur les pharmacies et les pharmacies de garde, tout en proposant un espace de gestion pour les administrateurs et une assistance conversationnelle pour les premiers secours. La solution s'appuie sur une API backend, un tableau de bord administrateur, une application mobile et un assistant de premiers secours basé sur la récupération d'informations hybride.

Résultats obtenus

Les objectifs principaux ont été atteints. L'utilisateur mobile peut rechercher des pharmacies, utiliser la géolocalisation, consulter les pharmacies de garde par date et accéder au catalogue de médicaments. Il peut également consulter l'assistant de premiers secours pour obtenir des réponses fiables et sûres à des questions d'urgence médicale. L'administrateur peut gérer les pharmacies, les gardes, les médicaments, les utilisateurs autorisés, les imports CSV, les indicateurs et les journaux d'audit. Le backend assure la centralisation des données, l'authentification, le contrôle des rôles, la validation des API et la traçabilité des actions sensibles. L'assistant offre une récupération précise d'informations avec un Recall@5 de 94.38%, une fusion hybride BM25/sémantique et des mécanismes de confiance et de sécurité pour éviter les hallucinations et ajouter les mises en garde appropriées pour les urgences.

Sur le plan technique, ce travail a mobilisé plusieurs compétences : conception d'API REST, développement backend avec FastAPI, modélisation SQLAlchemy, développement web avec React et Vite, développement mobile avec Expo et React Native, architecture de récupération d'informations hybride, entraînement et optimisation de pipelines NLP, authentification JWT, validation de fichiers CSV, journalisation, indica-

teurs et tests automatisés. Il a également montré l'intérêt d'une séparation claire entre les routes, les services, les modèles, les interfaces clientes, la base de données et les services spécialisés comme l'assistant.

Limites

Certaines limites restent à prendre en compte. Le catalogue des médicaments ne représente pas encore le stock réel de chaque pharmacie. Les données de garde dépendent toujours de la qualité des fichiers importés. L'assistant, bien que performant sur la récupération, pourrait bénéficier d'une amélioration de la sélection de spans ($F1 = 0.2990$ contre une cible de 0.35), possibilité de réentraînement avec des données d'entraînement enrichies pour affiner davantage l'extraction. Les tests bout en bout et les tests de charge doivent encore être renforcés. L'application mobile devra aussi être testée sur davantage d'appareils et dans différents contextes de connexion. L'intégration de l'assistant au sein de l'application mobile nécessite des tests supplémentaires pour assurer la cohérence de l'expérience utilisateur.

Perspectives

Les perspectives d'amélioration sont les suivantes :

- ajouter la disponibilité des médicaments par pharmacie ;
- permettre aux pharmacies de mettre à jour directement certaines informations sous contrôle ;
- envoyer des notifications lorsque les gardes changent ;
- améliorer la recherche par synonymes, erreurs de frappe ou noms commerciaux proches ;
- améliorer l'extraction de spans de réponse de l'assistant grâce à un reclassement plus fin ou un rétiquetage des données ;
- étendre la base de connaissance de l'assistant à d'autres sujets médicaux pertinents pour les pharmacies ;
- ajouter des capacités multilingues robustes à l'assistant (tests sur l'arabe et l'anglais) ;
- intégrer des tests bout en bout couvrant l'application mobile, le tableau de bord, l'API et l'assistant ;
- renforcer le déploiement par une configuration de production, une supervision, des sauvegardes et un monitoring de l'assistant ;

- enrichir les indicateurs pour mieux suivre les recherches, la couverture territoriale et l'utilisation de l'assistant ;
- construire une interface d'administration pour gérer les données d'entraînement et les performances de l'assistant.

Ainsi, PharmacieConnect constitue une base fonctionnelle pour une solution complète de gestion pharmaceutique avec assistance intelligente. Le projet peut encore évoluer, mais il montre déjà qu'une architecture bien organisée, associée à des techniques modernes comme la récupération hybride d'informations, peut répondre à la fois aux besoins du citoyen, aux besoins de l'administration et aux besoins d'assistance médicale d'urgence.

Bibliographie

- [1] K. Schwaber and J. Sutherland, “The scrum guide.” <https://scrumguides.org/>, 2020. Guide consulté le 13 mai 2026.
- [2] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley, 2005.
- [3] Conseil National de l’Ordre des Pharmaciens de Tunisie, “Site officiel du cnopt.” <https://cnopt.tn/>, 2026. Consulté le 13 mai 2026.
- [4] Med.tn, “Pharmacie de garde.” <https://www.med.tn/pharmacie/garde>, 2026. Consulté le 13 mai 2026.
- [5] Google, “Google maps.” <https://www.google.com/maps>, 2026. Service consulté le 13 mai 2026.
- [6] OpenStreetMap Foundation, “Openstreetmap.” <https://www.openstreetmap.org/>, 2026. Service consulté le 13 mai 2026.
- [7] Conseil national de l’ordre des pharmaciens de Tunisie, “Apothicare app.” <https://apps.apple.com/tn/app/apothicare/id6760540666>, 2026. Page App Store consultée le 13 mai 2026.
- [8] O. Foundation, “Owasp top 10 web application security risks.” <https://owasp.org/www-project-top-ten/>, 2026. Référence générale consultée le 12 mai 2026.
- [9] M. Jones, J. Bradley, and N. Sakimura, “Json web token.” <https://www.rfc-editor.org/rfc/rfc7519>, 2015. RFC 7519.
- [10] FastAPI, “Fastapi documentation.” <https://fastapi.tiangolo.com/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [11] M. O. Source, “React documentation.” <https://react.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [12] Vite, “Vite documentation.” <https://vite.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [13] Expo, “Expo documentation.” <https://docs.expo.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.

- [14] M. O. Source, “React native documentation.” <https://reactnative.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [15] SQLAlchemy, “Sqlalchemy documentation.” <https://docs.sqlalchemy.org/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [16] P. G. D. Group, “Postgresql documentation.” <https://www.postgresql.org/docs/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [17] OpenStreetMap Foundation, “Nominatim usage policy.” <https://operations.osmfoundation.org/policies/nominatim/>, 2026. Politique d’utilisation consultée le 13 mai 2026.
- [18] Pytest, “Pytest documentation.” <https://docs.pytest.org/>, 2026. Documentation consultée le 13 mai 2026.
- [19] M. O. Source, “Jest documentation.” <https://jestjs.io/docs/getting-started>, 2026. Documentation consultée le 13 mai 2026.
- [20] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

Annexes

A.1 Descriptions complémentaires des cas d'utilisation

Cette annexe regroupe des éléments complémentaires utiles à la lecture du rapport : correspondance avec les cas d'utilisation, emplacements réservés aux visuels, commandes de lancement, commandes de test, résultats de validation et exemples de structure CSV.

TABLEAU A.1 – Cas d'utilisation et éléments de validation

Cas d'utilisation	Complément attendu
Authentification	Décrite par un cas d'utilisation textuel et un diagramme de séquence JWT au chapitre Conception.
Recherche de pharmacie	Décrite par un cas d'utilisation textuel, un diagramme de séquence et une vue mobile de recherche.
Consultation des gardes	Décrite dans les besoins fonctionnels, les intégrations API et la vue mobile du calendrier.
Gestion des pharmacies	Décrite dans les besoins administrateur, la réalisation web et la vue de gestion des pharmacies.
Import CSV	Décrit dans la réalisation backend, la stratégie de validation et les exemples de structure CSV.
Audit logs	Décrit dans la sécurité, les tests backend et les indicateurs du tableau de bord.

A.2 État des captures et diagrammes

TABLEAU A.2 – Éléments visuels à insérer

Élément	Statut
Diagramme de cas d'utilisation	Emplacement réservé ; insérer le diagramme final exporté.
Diagramme de classes	Emplacement réservé ; insérer le diagramme final exporté.
Diagramme de séquence authentification JWT	Emplacement réservé ; insérer le diagramme final exporté.
Diagramme de séquence recherche pharmacie	Emplacement réservé ; insérer le diagramme final exporté.
Diagramme d'activité import CSV	Emplacement réservé ; insérer le diagramme final exporté.
Diagramme de composants	Emplacement réservé ; insérer le diagramme final exporté.
Diagramme de déploiement	Emplacement réservé ; insérer le diagramme final exporté.
Planning de réalisation	Emplacement réservé ; insérer le planning final exporté.
Capture tableau de bord administrateur	Emplacement réservé ; insérer la capture réelle.
Capture gestion des pharmacies	Emplacement réservé ; insérer la capture réelle.
Capture accueil mobile	Emplacement réservé ; insérer la capture réelle.
Capture calendrier des gardes	Emplacement réservé ; insérer la capture réelle.
Capture catalogue médicaments	Emplacement réservé ; insérer la capture réelle.

A.3 Commandes de lancement

Backend

```
cd backend_pharmacie
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload
```

Interface d'administration

```
cd admin_pharmacie
npm install
npm run dev
```

Application mobile

```
cd ouerkema-pharmacieconnect-4c94773cce7d
npm install
npm start
```

A.4 Commandes de test

Backend

```
cd backend_pharmacie
source venv/bin/activate
pytest
```

```
python -m pytest tests/test_analytics_dashboard.py \
    tests/test_assistant_regions.py \
    tests/test_audit_logs.py --tb=short
```

Administration web

```
cd admin_pharmacie
npm run lint
npm run build
```

Application mobile

```
cd ouerkema-pharmacieconnect-4c94773cce7d
npm test
npm run lint
```

A.5 Résultats de validation du 13 mai 2026

TABLEAU A.3 – Résultats détaillés des commandes exécutées

Commande	Objectif	Résultat
<code>pytest</code> ciblé backend	Vérifier les indicateurs, l'audit et les restrictions régionales	14 tests réussis, 21 avertissements, 4,45 s.
<code>npm run build</code>	Vérifier la génération de l'administration web	Build réussi, artefacts générés dans <code>dist/</code> .
<code>npm test -runInBand</code>	Vérifier les tests unitaires mobiles	7 tests réussis sur la recherche mobile par localisation.
<code>pytest</code> upload médicaments	Vérifier l'import des médicaments	Timeout après 20 s ; correction ou isolation nécessaire avant validation exhaustive.

A.6 Variables d'environnement

TABLEAU A.4 – Variables d'environnement utilisées par le backend

Variable	Rôle
<code>DATABASE_URL</code>	Chaîne de connexion à la base de données.
<code>SECRET_KEY</code>	Clé de signature des jetons JWT.
<code>ALGORITHM</code>	Algorithme de signature des jetons.
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	Durée de validité du jeton d'accès.
<code>REFRESH_TOKEN_EXPIRE_DAYS</code>	Durée de validité du jeton de rafraîchissement.
<code>FRONTEND_URL</code>	Origine autorisée pour l'interface web.
<code>TRUSTED_HOSTS</code>	Hôtes autorisés par le backend.
<code>REDIS_URL</code>	URL du service Redis lorsque le cache est utilisé.
<code>ENABLE_REDIS_CACHE</code>	Active ou désactive le cache Redis si configuré.

A.7 Exemples CSV

Pharmacies

TABLEAU A.5 – Exemple de structure CSV pour les pharmacies

Colonne	Description
name	Nom de la pharmacie.
latitude	Latitude GPS.
longitude	Longitude GPS.
address	Adresse complète ou approximative.
phone	Numéro de téléphone.
governorate	Gouvernorat utilisé pour le filtrage régional.

Médicaments

TABLEAU A.6 – Exemple de structure CSV pour les médicaments

Colonne	Description
code_pct	Identifiant métier du médicament.
nom_commercial	Nom commercial.
prix_public_dt	Prix public en dinars.
tarif_reference_dt	Tarif de référence.
categorie_remboursement	Catégorie de remboursement.
dci	Dénomination commune internationale.
ap	Autorisation ou indicateur AP selon la source de données.